

Dedicated to my family and friends for their unconditional love, care, support
and continuous encouragement throughout my life.

Acknowledgements

All praises to Allah for the strengths and His blessings throughout my life.

All dreams require belief to come to fruition, and for that, I owe immense gratitude to my supervisor, Dr. M. Naeem Ali Bhatti. His unwavering support, guidance, and motivation have been invaluable throughout the course of my dissertation. Without his influence, venturing into this domain would have been unimaginable. I am profoundly thankful for his consistent assistance and encouragement during my Bachelor's studies. Dr. Bhatti's insights and discussions have been remarkably enlightening, extending far beyond academics into every aspect of life. I will always cherish his calm, tolerant, and down-to-earth nature, which has left a lasting impression on me.

I am deeply indebted to the faculty members of the Department of Electronics at QAU, especially Dr. Zia, Dr. Qaisar Abass Naqvi, and Dr. Aqueel Ashraf. Their wisdom, care, and motivational support have been instrumental in boosting my morale throughout my studies.

Special thanks go to the team at AiTEC (Artificial Intelligence Technology Centre, National Centre For Physics Islamabad), particularly Dr. Khursheed Hasnain (Director General AiTEC). I am also incredibly grateful to the AMs at NLP and SoEC Lab, including Awais Akhtar, Muhammad Zain Tariq, Muhammad Saleh, and Hanif, for their continuous guidance and assistance, which made this project possible.

I also extend my heartfelt thanks to Urooj, Mian Hassan Azhar, Ahmed Aslam, Daniyal Khan Durani, Talha, Rana Naveed , Hamxa Kashif, Mangi, Muneeb Junior, Zain, for their immeasurable kindness, love, and support. Your companionship and encouragement have been greatly appreciated.

Finally, my deepest gratitude goes to my parents for their endless love and efforts that have enabled me to reach this remarkable position in life.

Muhammad Sohaib
February, 2025

Abstract

The need for disease detection in agriculture is critical, as plant diseases can lead to significant yield losses, reduced crop quality, and increased production costs. Early and accurate detection enables timely intervention, minimizing damage and supporting sustainable farming practices. Similarly, the ability to distinguish between crops and weeds is essential for optimizing resource usage and improving crop yields. Weeds compete with crops for nutrients, water, and sunlight, often leading to reduced productivity and increased management costs. Effective weed detection allows for targeted removal, reducing the need for chemical herbicides and promoting more sustainable farming methods. This thesis presents the development and deployment of an autonomous agricultural drone designed for advanced farming applications. The drone is built on the Pixhawk flight controller and integrated with a Raspberry Pi 4, utilizing the Realtime Operating System (ROS) and MAV Link protocol for seamless communication and control. The drone supports advanced flight modes, including loiter, position hold, and stabilize, and is equipped with failsafe mechanisms such as auto-landing, return-to-launch, low battery, crash landing, and out-of-range detection, ensuring reliable and safe operation. The primary objective of this project is to enhance agricultural productivity through advanced crop monitoring techniques. The drone autonomously distinguishes between crops and weeds, detects diseases in wheat, rice, and various other crops, and identifies wheat heads for yield estimation. A robust machine learning pipeline, powered by a dataset encompassing 38 classes, facilitates the detection of plant diseases across multiple species, including apples,

blueberries, cherries, corn, grapes, oranges, peaches, peppers and more. The integrated camera captures detailed imagery for real-time analysis, allowing precise identification of crop health and potential threats. This autonomous system is designed to operate efficiently in diverse agricultural environments, providing farmers with actionable insights for targeted interventions.

Contents

Contents	vii
List of Figures	x
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Project Scope	2
1.5 Thesis Structure	3
2 Literature Review	4
2.1 Agricultural Drones and Autonomous Systems	4
2.2 Object Detection and Machine Learning in Agriculture	5
2.2.1 YOLOv5 And Tensor Flow Framework	6
2.3 Autonomous Navigation and Sensor Integration	9
2.4 Limitations of Existing Techniques	9
2.5 Summary	11
3 Methodology	13
3.1 System Architecture	13
3.1.1 Overview of the Project Framework	14
3.2 Hardware Components	17
3.2.1 Pixhawk	17
3.2.2 GPS	23

CONTENTS

3.2.3	BLDC Motor	23
3.2.4	TeraRanger.....	24
3.2.5	Transmitter and Receiver.....	25
3.2.6	Lipo Battery.....	26
3.2.7	Telemetry	27
3.2.8	Power Module XT60 90A Specifications	28
3.3	Integration of Components	29
3.3.1	Power Supply And Connectivity.....	29
3.3.2	Calibration, Firmware Script and Processing of Data.....	30
3.3.3	Source Code Scripts:.....	31
3.3.4	Circuit Diagram.....	31
3.4	Software in the Loop	33
4	Implementation	40
4.1	Dataset Collection and Annotation	40
4.1.1	Training Process and Parameters	44
4.1.2	TensorFlow Lite Conversion	47
4.1.3	Prediction and Evaluation.....	47
4.1.4	Visualizing the F1 Scores.....	47
4.1.5	Custom Image Loading and Prediction.....	48
4.1.6	Visualizing Wrong Predictions.....	49
4.1.7	Predictions Using Live Camera.....	50
4.2	Training Using Yolov5	50
4.2.1	Flow For A YOLOv5 Object Detection Model.....	50
4.3	Deploying a YOLOv5Model on a Raspberry Pi4.....	52
5	Experimental Results	53
5.1	Experimental Setup And Performance Metrics.....	53
5.2	Results Analysis	54
5.3	Discussion	64
5.3.1	Effectiveness of Datasets.....	64
5.3.2	Model Performance	64
5.3.3	Hardware-Software Integration.....	64

CONTENTS

5.4	Challenges and Limitations.....	65
5.4.1	Dataset Limitations.....	65
5.4.2	Hardware Constraints.....	65
5.4.3	Real-Time Processing.....	65
5.4.4	Safety and Reliability.....	66
5.5	Summary.....	66
6	Conclusion	67
6.1	Summary of Findings.....	67
6.1.1	Effective Object Detection	67
6.1.2	Real-Time Performance	67
6.1.3	Challenges and Limitations	68
6.2	Future Work	68
6.2.1	Dataset Expansion	68
6.2.2	Enhanced Hardware	69
6.2.3	Advanced Algorithm	69
6.2.4	Autonomous Decision-Making	69
	Bibliography	70

List of Figures

2.1	YOLOv5 Network-Architecture	8
3.1	System Pipeline.....	14
3.2	Flight Controller [8].	17
3.3	Introduction of Pixhawk	18
3.4	Pixhawk Interfaces	18
3.5	Pixhawk different pin configurations are given.	20
3.6	Drone Frame F450.....	20
3.7	Drone Components.	21
3.8	Drone Components.	22
3.9	TeraRanger	25
3.10	Firmware And Live Calibration.....	30
3.11	Drone Connection Diagram:.....	32
3.12	Teraranger connection with Pixhawk.....	33
3.13	Software in the loop.	34
3.14	Build environment.....	35
3.15	FlightGear 3D.....	37
3.16	Rpi4 With Pixhawk.	38
3.17	Enable Serial Port.	39
3.18	Mavproxy Commands.....	39
4.1	Datasets description is given in this Table.....	44
4.2	Flow-Chart.....	47
4.3	Bar Chart.....	48
4.4	Visualizing Wrong Predictions With Highest Probability.....	49

LIST OF FIGURES

4.5	Visualizing Wrong Predictions With Highest Probability.....	49
4.6	Real-time Inference	50
5.1	Real-Time Predictions Using YOLOv5 Model	56
5.2	Real-Time RICE Disease Detection.....	57
5.3	Real-Time RICE Disease Detection.....	58
5.4	Real-Time RICE DISEASE DETECTION	59
5.5	Real-Time Weed Vs Crop Detection	60
5.6	Real-Time Weed Vs Crop Detection	61
5.7	Wheat Disease Detection	62
5.8	Wheat Disease Detection	63

List of Acronyms

AHRS	Attitude and Heading Reference System
AFHDS	Automatic Frequency Hopping Digital System
BLDC	Brushless Direct Current (Motor)
CEP	Circular Error Probable
DSC	Digital Signal Controller
EGNOS	European Geostationary Navigation Overlay Service
FS	FlySky (Brand)
GFSK	Gaussian Frequency Shift Keying
GHz	Gigahertz (Frequency Units)
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
IoT	Internet of Things
LiPo	Lithium Polymer (Battery)

LIST OF FIGURES

MAVLink Micro Air Vehicle Communication Protocol

MHz Megahertz (Frequency Units)

MSAS Multi-functional Satellite Augmentation System

PPM Pulse Position Modulation

PX4 Open-source Flight Control Software

QZSS Quasi-Zenith Satellite System

PID Proportional-Integral-Derivative (Controller)

ROS Robot Operating System

RPM Revolutions Per Minute

RTL Return to Launch (Drone)

SBAS Satellite-Based Augmentation System

ToF Time of Flight

UAV Unmanned Aerial Vehicle

USB Universal Serial Bus

VDC Volts Direct Current

Chapter 1

Introduction

1.1 Background

Agriculture is a vital sector that sustains the global population, but it faces numerous challenges, including the need for increased productivity, efficient resource management, and the mitigation of crop losses due to diseases and weeds. Traditional farming methods, which rely heavily on manual labor and chemical inputs, are often inefficient and can have detrimental environmental effects. The advent of autonomous systems and machine learning has opened new possibilities for precision agriculture, where technology plays a crucial role in optimizing crop management and improving yields. This project aims to leverage these advancements by developing an autonomous agricultural drone capable of detecting crops, weeds, and plant diseases, thereby enhancing agricultural practices through real-time data analysis and targeted interventions.

1.2 Problem Statement

Farmers often face significant challenges in managing weeds, detecting crop diseases, and estimating yields, all of which are crucial for maximizing productivity and sustainability. Manual monitoring of large fields is time-consuming, labor-intensive, and prone to human error, leading to suboptimal decision-

making. Furthermore, the indiscriminate use of chemical herbicides and pesticides can harm the environment and reduce crop quality. There is a pressing need for a system that can automate the monitoring and management of agricultural fields, providing accurate, real-time data on crop health, weed presence, and disease outbreaks. This project seeks to address these challenges by developing an autonomous drone system equipped with advanced sensors and machine learning algorithms to perform these tasks efficiently and effectively [1].

1.3 Objectives

The primary aim of this project is to develop an autonomous drone system capable of efficiently navigating agricultural fields by leveraging GPS and various sensors. The system integrates a machine learning pipeline designed to differentiate between crops and weeds, detect plant diseases, and identify wheat heads using tensorflow, yolov5 and tflite on Rpi4 [2]. The drone utilizes a Pixhawk flight controller and Raspberry Pi 4, with ROS and MAVLink protocols for seamless communication and control. To ensure reliable and safe operation, the drone is equipped with failsafe mechanisms such as auto-landing, return-to-launch, low battery detection, crash detection, and out-of-range detection. Additionally, the onboard camera facilitates real-time data analysis, enabling timely interventions and targeted management of agricultural fields. By enhancing agricultural efficiency and sustainability, the drone contributes to reducing the reliance on chemical inputs and minimizing crop losses through early detection of diseases and weed infestations..

1.4 Project Scope

This project encompasses the design, development, and testing of an autonomous agricultural drone system. It involves the integration of hardware components, including the Pixhawk flight controller, Raspberry Pi 4, GPS, TeraRanger range finder sensor, camera, and motors. Software development focused on

implementing ROS and MAVLink for flight control, alongside custom scripts for autonomous navigation and data processing. The project also includes the development of machine learning algorithms for weed detection, crop versus weed classification, plant disease detection across 38 crop classes, and wheat head identification. Field testing is conducted in both simulated environments, such as FlightGear, SITL and real agricultural settings to assess the system's performance, reliability, and accuracy. Safety features were incorporated, including auto-landing, return-to-launch and crash detection mechanisms [3].

1.5 Thesis Structure

This thesis is divided into sub chapters, each having information about different aspects of the project

Chapter 1: Introduction - This section summarizes the project's origins, the main issue it aims to solve, its goals, its focal point, and the way this thesis is structured.

Chapter 2: Literature Review - This section reviews existing research on agricultural drones, machine learning for crop and weed detection, and related technologies.

Chapter 3: Methodology – Details the development of the drone, including hardware integration, software implementation, ROS and MAVLink and the machine learning algorithms for detection tasks.

Chapter 4: Implementation – Describes the assembly and configuration of the drone, including the integration of components and the setup of systems for both simulation and real-world testing.

Chapter 5: Experimental Results–Summarizes the findings from testing the drone in various conditions and evaluates its performance in achieving the project's goals. This chapter also presents the results, discusses the project's limitations, and provides recommendations for future improvements.

Chapter 6: Conclusion –Presents the main outcomes of the project, reflecting on its achievements and impact on agricultural practices.

Chapter 2

Literature Review

The integration of robotics and artificial intelligence (AI) in agriculture has seen significant advancements, particularly in the realm of autonomous systems for crop and weed management, disease detection, and precision farming. This chapter reviews existing research on agricultural drones, object detection algorithms, autonomous navigation systems, and previous techniques, while highlighting the gaps that this project addresses

2.1 Agricultural Drones and Autonomous Systems

The use of drones in agriculture has evolved significantly over the past decade, driven by advances in robotics and AI. Autonomous drones have become crucial for monitoring large agricultural fields, optimizing crop management, and improving efficiency. Research by Zhang et al. (2017) discusses the benefits of using drones equipped with various sensors for crop health monitoring and precision agriculture [3]. Traditional methods, such as manual crop inspection and satellite imagery, have limitations in terms of cost, time, and resolution. Drones offer a more cost-effective and efficient alternative by providing high-resolution images and real-time data.

Earlier techniques for agricultural monitoring included the use of fixed-wing aircraft and satellites, which, while useful, lacked the flexibility and granularity of drone-based systems. Drones equipped with multispectral and hy-

perspectival sensors can now provide detailed insights into plant health and soil conditions, enabling more precise management decisions.

2.2 Object Detection and Machine Learning in Agriculture

Accurate object detection is crucial for the effective functioning of autonomous agricultural systems. The use of machine learning algorithms has significantly advanced the ability to monitor and manage crops and weeds efficiently.

The YOLO algorithm is one of the most notable methods for real-time object detection due to its speed and accuracy. YOLOv5, an enhanced version of this algorithm, offers superior performance with improved detection efficiency and reduced latency (Jocher et al., 2020). YOLOv5 has found applications in various domains, including robotics and surveillance, but its application in agriculture for tasks such as detecting crop diseases and distinguishing between crops and weeds is still developing. TensorFlow (TF), an open-source machine learning framework developed by Google, plays a vital role in the development and deployment of these algorithms. TensorFlow provides a robust ecosystem for building, training, and evaluating machine learning models. It supports a wide range of tasks, from basic classification to complex object detection. The framework's flexibility and scalability make it ideal for developing models that can handle diverse agricultural challenges [4].

TensorFlow Lite (TFLite), a lightweight version of TensorFlow, is designed for deploying models on mobile and embedded devices. For agricultural drones, TFLite is particularly useful because it enables real-time inference on devices with limited computational resources. By converting TensorFlow models into the TFLite format, the project ensures that complex algorithms can run efficiently on the drone's onboard hardware and on Rpi4 [5].

Earlier techniques for agricultural monitoring and management included manual visual inspections and traditional image processing methods, such as color-based segmentation and texture analysis. These methods, while useful,

were often labor-intensive, time-consuming, and subject to human error. The introduction of machine learning algorithms like YOLOv5 represents a significant improvement, as they automate the detection process, enhance accuracy, and enable real-time data processing.

In addition to YOLOv5, earlier object detection frameworks such as R-CNN and its variants (Fast R-CNN, Faster R-CNN) provided foundational techniques for object detection. While effective, these methods typically had higher computational demands and longer processing times compared to YOLOv5, making them less suitable.

This project aims to leverage YOLOv5 and TensorFlow frameworks to address the limitations of previous techniques by developing an autonomous agricultural drone capable of real-time detection of crop diseases, weeds, and wheat heads. The use of TensorFlow for model development and TensorFlow Lite for deployment ensures that the drone can perform complex object detection tasks efficiently on embedded hardware. By integrating these advanced technologies, the project enhances agricultural efficiency and supports more precise and timely management practices.

2.2.1 YOLOv5 And Tensor Flow Framework

- **Open-Source Library:** Developed by Google, TensorFlow is an open-source machine learning framework that supports a wide range of tasks from basic classification to complex neural networks [6].
- **Flexible Ecosystem:** Provides tools for building, training, and deploying machine learning models, including support for deep learning, reinforcement learning, and more.
- **Scalability:** Capable of handling large-scale data and complex models, making it suitable for high-performance computing environments.
- **Comprehensive API:** Offers APIs in Python, C++, Java, and other languages, allowing for integration with various applications and systems.

- **Model Training and Evaluation:** Facilitates the development and evaluation of custom machine learning models, including support for data pipelines, model training, and performance monitoring.

YOLOv5 is a state-of-the-art object detection algorithm for speed and accuracy, making it highly applicable in real-time applications [7]. When the system is given a frame, several critical steps describe the process it passes through which are:

- **Input Frame Processing:** The input frame, typically an RGB image, is first resized to a fixed dimension (usually 640x640 pixels). This resizing ensures consistency across various input sizes, facilitating faster and more reliable processing.
- **Feature Extraction:** The resized frame is then passed through a series of convolutional layers, which serve as feature extractors. These layers detect various features such as edges, textures, and patterns that are crucial for identifying objects within the frame.
- **Object Detection:** YOLOv5 uses a unified neural network to simultaneously predict bounding boxes and class probabilities from entire images in a single forward pass. The network partitions the image into a grid and, for each grid cell, generates several bounding boxes along with their associated confidence scores. These confidence scores indicate both the probability that a bounding box contains an object and the accuracy of the bounding box prediction.
- **Non-Maximum Suppression (NMS):** To refine the detection results, YOLOv5 applies non-maximum suppression, which eliminates redundant bounding boxes. This step ensures that each detected object is represented by only one bounding box, minimizing false positives and improving detection accuracy.
- **Output:** The final output consists of bounding boxes around detected objects, along with their class labels and confidence scores.

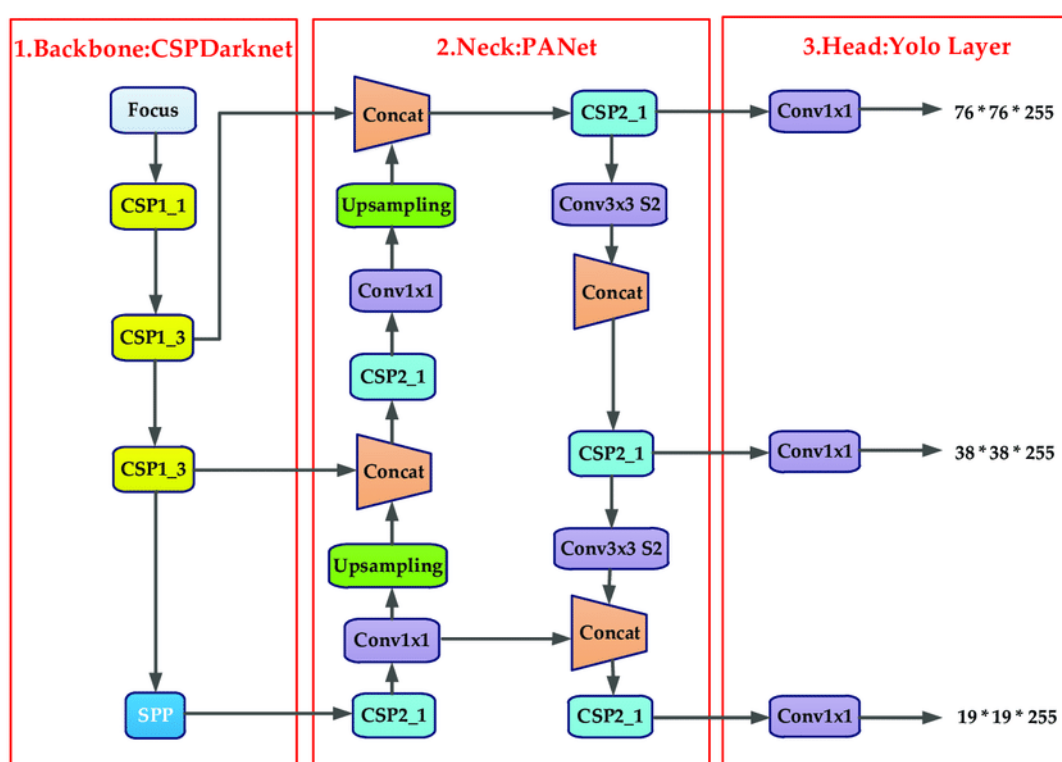


Figure 2.1: YOLOv5 Network-Architecture

2.3 Autonomous Navigation and Sensor Integration

Effective navigation and altitude measurements are crucial for autonomous agricultural drones. The project integrates several advanced technologies to achieve reliable operation and precise data collection. Unlike traditional methods that rely on remote controllers, this project utilizes GPS for positioning, a TeraRanger range finder for altitude measurements, gyro, accelerometer and 2 compasses for orientation. These components work together to ensure accurate navigation and real-time monitoring.

Previous studies, such as Zhang et al. (2019), explored the use of Lidar and visual sensors for (SLAM), enhancing navigation capabilities. The integration of GPS, a compass, and TeraRanger range finders addresses the limitations of conventional remote-controlled navigation systems by providing a more autonomous and responsive approach to handling dynamic agricultural environments.

2.4 Limitations of Existing Techniques

- **Real-Time Agricultural Detection:** While object detection algorithms like YOLO have demonstrated effectiveness in various applications, their adaptation for specific agricultural tasks such as detecting plant diseases, differentiating between crops and weeds, and identifying wheat heads is still under development. The unique challenges posed by diverse plant species and environmental conditions in agriculture necessitate further refinement of these technologies for optimal performance in real-world scenarios.
- **Autonomous Navigation in Agriculture:** The field of autonomous navigation for agricultural drones is progressing, but there is still a need for enhanced capabilities to handle the complexities of dynamic agricultural environments. Current research often lacks comprehensive solutions that integrate modern sensors, such as range finders and advanced GPS systems, with robust navigation algorithms to ensure reliable operation in

varying conditions.

- **Comprehensive Agricultural System Integration:** Existing approaches often falls short of providing a fully integrated system that combines accurate object detection, advanced autonomous navigation, and real-time data analysis. There is a need for a holistic approach that integrates these components seamlessly to improve agricultural management efficiency, allowing for timely interventions and precise monitoring of crops and environmental conditions.

Silent Features of the Proposed Framework

Flight Control:

- **Description:** An advanced flight controller that integrates with various sensors and communication systems to provide precise control and autonomous navigation.
- **Advantages:** Supports advanced flight modes, integration with multiple sensors, and customizable control settings.

Seamless Control and Data Exchange:

- **Description:** Utilizes a Raspberry Pi 4 for processing and communication, with ROS (Robot Operating System) and MAVLink for seamless control and data exchange.
- **Advantages:** Enhances computational capabilities and enables real-time data processing and communication between the flight controller and other systems.

Range Finder:

- **Description:** Provides distance measurements to detect obstacles and navigate complex environments.
- **Advantages:** Improves obstacle avoidance and allows for precise navigation in dynamic environments.

Advanced Flight Modes:

- **Description:** Includes modes like Loiter, Position Hold, and Stabilize, with enhanced capabilities for autonomous navigation and adaptive responses to environmental changes.
- **Advantages:** Offers flexibility and improved safety by dynamically adjusting flight behavior based on real-time conditions.

Auto-Landing and Return-to-Launch (RTL):

- **Description:** Advanced features for automatic landing and returning to the launch point, with additional safety mechanisms for low battery and crash detection.
- **Advantages:** Ensures reliable operation and safe recovery of the drone in various scenarios.

Integration with Simulation Tools (e.g., FlightGear, SITL):

- **Description:** Uses simulation tools to test and refine navigation and sensor integration before deployment in real environments.
- **Advantages:** Allows for comprehensive testing and validation of autonomous behaviors in controlled conditions.

These advancements represent a significant evolution from traditional methods, providing more robust, adaptive, and reliable navigation and sensor integration for autonomous agricultural drones.

2.5 Summary

In this project, we attempt to overcome the limitations of the existing frameworks by developing an autonomous agricultural drone equipped with TF, YOLOv5 for real-time detection of crop diseases, weeds, and wheat heads. The integration of advanced sensors, autonomous navigation capabilities, and

LITERATURE REVIEW

machine learning algorithms, along with TensorFlow and TensorFlow Lite for efficient model deployment, aims to enhance agricultural efficiency and sustainability. By automating the detection and classification processes, the drone reduces the need for manual inspection, minimizes crop losses, and supports more targeted management practices. This project represents a significant advancement in precision agriculture, offering a comprehensive solution to modern farming challenges.

Chapter 3

Methodology

This chapter describes the proposed approach used in the creation and application of the Autonomous Agricultural Drone. We focus on the establishment of the system and its components, including system's hardware and software, data augmentation and pre-processing, the training of the object detection model and the overall assembling of the system. This chapter also incorporates the methods, tools, and algorithms applied, in addition to the mode of data collection and analysis processes.

3.1 System Architecture

The drone is designed in a way that it first performs arming pre-checks like AHRS, GPS lock, telemetry, rangefinder, battery voltages, compasses alignment and other sensors to see if everything is working satisfactorily. Then mission planner as well as Qground that are integrated with the drone give a green signal indicating the drone is ready to fly otherwise a red signal is generated (with corresponding error messages given in user interface and in log file). For autonomous navigation, the drone setup seeks an input from the user to mark the location provided by GPS on screen and make waypoints to move the drone in that particular filed. Subsequently, the drone takes off and moves to all those coordinates given by user one by one by running path planning algorithms. Then the camera attached to Rpi4 captures the video and

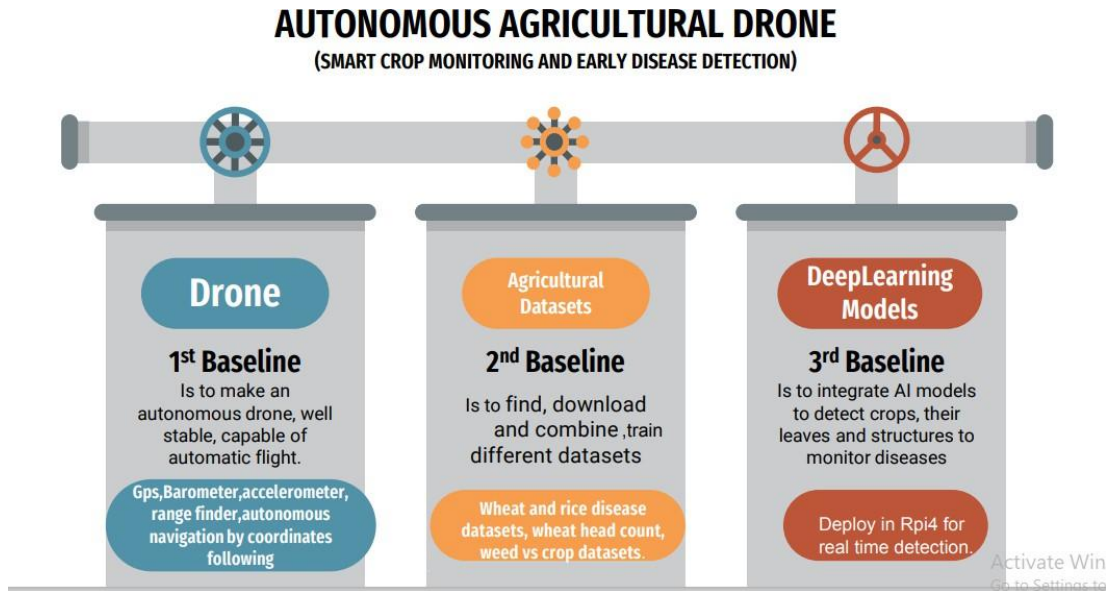


Figure 3.1: System Pipeline.

runs deep learning algorithm by using ssh command and processes the video frames showing real time video on local IP host and saves the video.

3.1.1 Overview of the Project Framework

Our proposed project framework consists of the following modules.

Pre-Flight Checks:

- **AHRS (Attitude and Heading Reference System):** verifies the drone's orientation and stability.
- **GPS Lock:** ensures that the GPS module has acquired a sufficient number (range 9 to 12) of satellites for accurate positioning.
- **Telemetry:** checks communication between the drone and the ground control station (mission planner or Qground).
- **Rangefinder:** confirms that the altitude measuring sensor is functioning correctly.

- **Battery Voltage:** monitors battery levels to ensure there is sufficient power for the flight.
- **Compass Alignment:** validates the calibration and alignment of the compass to prevent navigation errors.
- **Test Sensors:** tests additional sensors for functionality and integration.

UI Integration (Mission Planner/QGroundControl):

- **Error Messages and Status Indicators:** the UI displays error messages if any pre-check fails or a green signal if all systems are ready for flight.
- **Waypoint Creation:** users can mark GPS locations on the screen and set waypoints for the drone to follow in the field.

Flight Operations:

- **Automated Takeoff:** the drone initiates takeoff (5m above the ground) based on the pre-flight checks and stabilizes in the air.
- **Waypoint Navigation:** the drone follows the predefined waypoints using path planning algorithms to navigate through the field efficiently.

Data Capture and Processing:

- **Video Capture:** the camera attached to the Raspberry Pi4 captures real-time video footage of the agricultural field.

Deep Learning Processing:

- **Remote Command Execution:** using SSH commands, the Raspberry Pi4 runs deep learning algorithms to process the video frames.
- **Real-Time Video Processing:** the video frames are processed to identify plant diseases, distinguish between crops and weeds, and detect wheat heads.

- **Local IP Hosting:** real-time video is displayed on a local IP host for monitoring and analysis.
- **Data Storage:** processed video frames and detection results are saved in memory for further analysis and record-keeping.

Data Review:

- The collected video footage and analysis results are reviewed to assess the performance of the drone and the accuracy of the detection algorithms.
- **System Diagnostics:** the post-flight checks are performed to ensure the drone's components are functioning correctly and to identify any issues that need addressing.

Overview of Main Components: Detection and identification tasks are achieved through the YOLOv5 and TF models, which have been trained on five different datasets. In the following, we give the working of different components on the drone to detect wheat, rice and other diseases.

- **SSH Command:** Rpi4 is connected to PC through ssh and is given a command to capture video and run inference.
- **Camera:** Records what is happening in the environment and passes the video feed to the Rpi4.
- **Rpi4:** Uses the YOLOv5 and TF models to detect objects from the video stream input.
- **Streaming:** Rpi4 then performs detection and streams video to local IP address and also saves it to memory.
- **Drone:** Keeps following given waypoints by provided speed and altitude and returns back to launch position and disarms.

3.2 Hardware Components

In this section, we provide the used hardware components with their visuals for better understanding of pin details.

3.2.1 Pixhawk

The used flight controller is called as Pixhawk 2.4.8 is given in Figure 3.2.

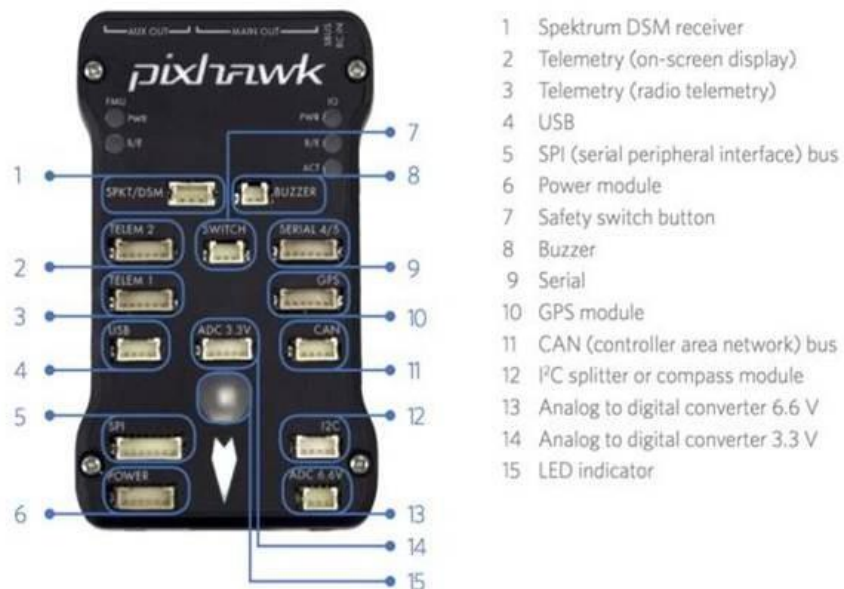


Figure 3.2: Flight Controller [8].

Introduction

Pixhawk is a family of open-source autopilots designed for unmanned aerial vehicles (UAVs), commonly known as drones.



Figure 3.3: Introduction of Pixhawk

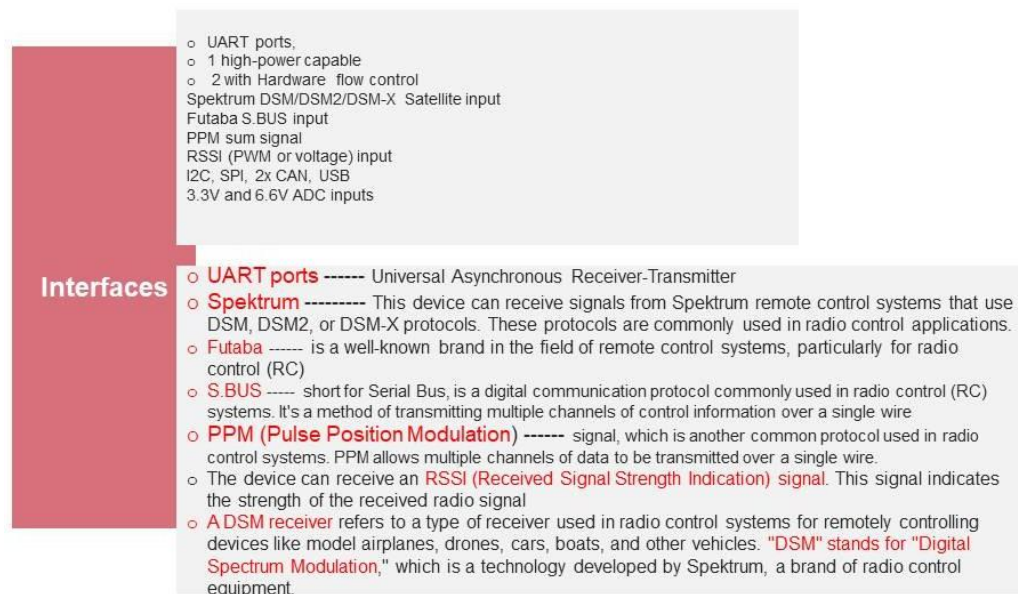


Figure 3.4: Pixhawk Interfaces

Pixhawk connector pin assignments

TELEM1, TELEM2 ports

Pin	Signal	Volt
1 (red)	VCC	+5V
2 (blk)	TX (OUT)	+3.3V
3 (blk)	RX (IN)	+3.3V
4 (blk)	CTS	+3.3V
5 (blk)	RTS	+3.3V
6 (blk)	GND	GND

Pixhawk connector pin assignments

GPS port

Pin	Signal	Volt
1 (red)	VCC	+5V
2 (blk)	TX (OUT)	+3.3V
3 (blk)	RX (IN)	+3.3V
4 (blk)	CAN2 TX	+3.3V
5 (blk)	CAN2 RX	+3.3V
6 (blk)	GND	GND

SPI			POWER		
Pin	Signal	Volt	Pin	Signal	Volt
1 (red)	VCC	+5V	1 (red)	VCC	+5V
2 (blk)	SPI_SCK	3.3V	2 (blk)	VCC	+5V
3 (blk)	SPI_MISO	+3.3V	3 (blk)	CURRENT	up to +3.3V
4 (blk)	SPI_MOSI	+3.3V	4 (blk)	VOLTAGE	up to +3.3V
5 (blk)	!SPI_NSS	+3.3V	5 (blk)	GND	GND
6 (blk)	!GPIO	+3.3V	6 (blk)	GND	GND
7 (blk)	GND	GND			

Figure 3.5: Pixhawk different pin configurations are given.



Figure 3.6: Drone Frame F450.

METHODOLOGY

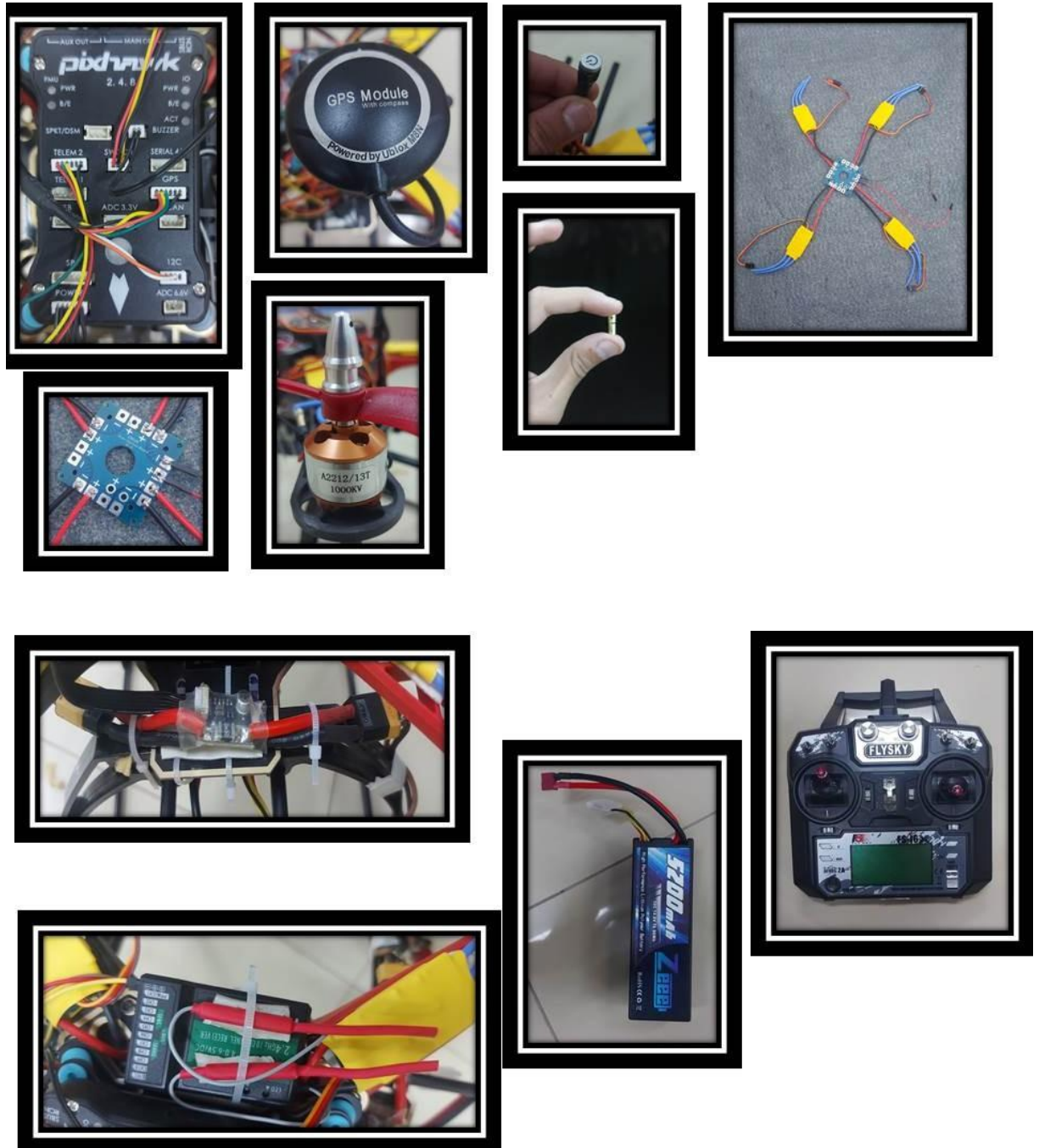


Figure 3.7: Drone Components.

METHODOLOGY

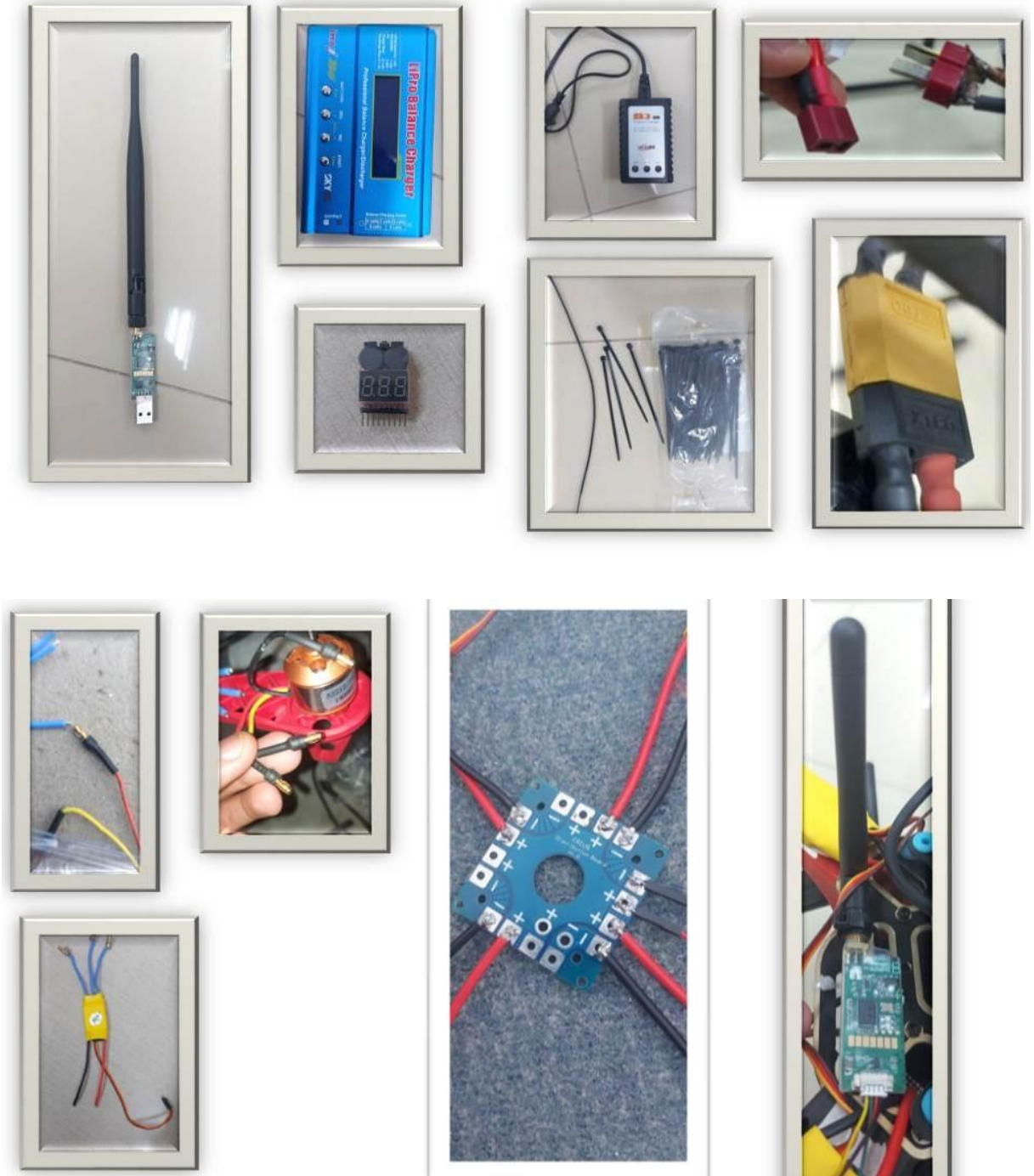


Figure 3.8: Drone Components.

3.2.2 GPS

The new M8N GPS module offers improved accuracy and lower power consumption compared to its predecessor, the 7N module. With a positioning accuracy of up to 0.6 meters, it supports a variety of satellite systems including GPS/QZSS L1 C/A, GLONASS L10F, and BeiDou B1. The module is compatible with most flight controllers that use the NEO-M8N connector, such as Pixhawk, and includes a 72-channel u-blox M8 engine, which allows for versatile and high-precision navigation across different environments. The visual detail of M8N GPS module is given in Figure 3.7. The specifications are as follows.

- Frequency in single mode: up to 18Hz
- GNSS update frequency in multiple simultaneous modes: up to 10Hz
- Accuracy: 2.0 m CEP
- Fast re-acquisition time: 1.5 seconds
- Cold start: 26 seconds
- AssistNow GNSS Online/Offline support
- Multiple Constellation Support
- Dynamic Modes
- Low Power Consumption

3.2.3 BLDC Motor

The A2212 BLDC motor is designed for high-speed applications like drones and quadcopters. With a 22mm diameter and 12mm height, it provides a balance between speed and torque, suitable for a range of aerial applications. The motor has 13 turns per pole, translating to high revolutions per volt, which is critical for drones that require rapid maneuvers while still being able to lift significant weight. The visual detail of A2212 BLDC motor is given in Figure 3.7. The specifications of A2212 BLDC motor are as follows.

- RPM per volt: 1000
- Optimal for high torque and weight lifting
- **Cooling Mechanism:** The motor is equipped with a built-in cooling fan that dissipates heat, allowing for sustained high-performance without overheating.
- **Customizability:** Users can adjust parameters like timing and PWM (Pulse Width Modulation) to optimize performance for specific tasks.
- **Durability:** The motor's robust construction enables it to withstand the vibrations and shocks commonly encountered in aerial operations.
- Turns per pole: 13

3.2.4 TeraRanger

The TeraRanger Evo 60m sensor provides accurate distance measurements based on the Time-of-Flight principle. It is particularly suited for navigation in drones operating at low altitudes or indoors, where traditional GPS might fail. The sensor's ability to provide real-time distance data supports advanced functions such as obstacle avoidance and precision landing. The visual detail of TeraRanger Evo 60m is given in Figure 3.9. The specifications of TeraRanger Evo 60m sensor are as follows.

- **High Frame Rate:** The sensor offers a high frame rate, enabling it to provide continuous distance updates, which is essential for real-time navigation.
- **Compact Design:** Its lightweight and compact form factor make it easy to integrate into small drones without significantly affecting their flight dynamics.
- **Multiple Interfaces:** The sensor supports various communication interfaces, including I2C and UART, ensuring compatibility with different flight controllers.

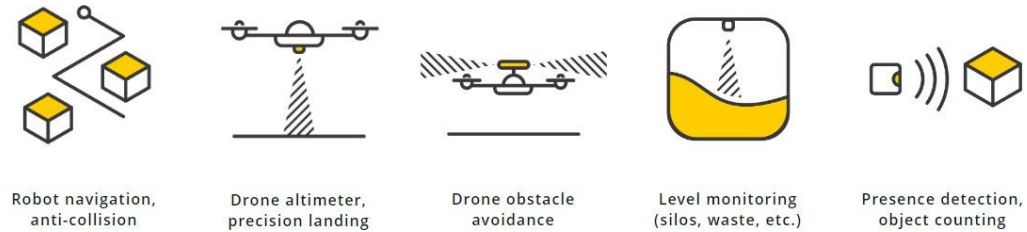


Figure 3.9: TeraRanger.

- Detection range: 50m

3.2.5 Transmitter and Receiver

Flysky FS-i6 6ch 2.4ghz Transmitter with FS-iA6B PPM Receiver The visual detail of Flysky FS-i6 is given in Figure 3.7. The specifications of Flysky FS-i6 are as follows.

Transmitter:

- **Model Types Supported:** The FS-i6 transmitter can be used with gliders, helicopters, and airplanes.
- **Frequency Band:** Operates on the 142 band.
- **System Compatibility:** Supports both AFHDS 2A and AFHDS protocols.
- **Signal Encoding:** Uses GFSK (Gaussian Frequency-Shift Keying) for reliable communication.
- **Signal Output:** Provides a PPM output for connecting to other devices or systems.
- **Charger Port:** Does not include a built-in charging port, requiring external battery charging.
- **Antenna Configuration:** Equipped with dual 26mm antennas.

- **Online Firmware Updates:** Capable of receiving updates online, ensuring it remains up-to-date.
- **Certifications:** Certified with CE0678 and FCC.
- **Model Memory:** Can store up to 20 different model settings.

Receiver:

- **Channel Count:** Supports up to 6 channels for controlling various functions of the aircraft.
- **Operating Frequency Range:** Works within the 2.4055–2.475 GHz frequency range.
- **Transmitting Power:** Outputs a transmitting power of 20 dBm, ensuring a strong and stable signal.
- **Receiver Sensitivity:** Has a high RF sensitivity of -105 dBm, allowing it to pick up weak signals effectively.
- **2.4G Mode:** Utilizes an advanced version of the automatic FM digital system, enhancing reliability and signal clarity.
- **Input Power Range:** Operates within a voltage range of 4.0 to 8.4 VDC, drawing up to 2A

3.2.6 Lipo Battery

The visual detail of Lipo Battery is given in Figure 3.7. The specifications of Lipo Battery are as follows.

- Zeee 4S 5200mAh LiPo Battery Specifications
- 14.8V
- 4 cells in series
- Capacity: 5200mAh

- 60C Discharging Rate
- Connector Type: Deans T Plug

3.2.7 Telemetry

The visual detail of 3DR Radio Telemetry System is given in 3.8. The specifications of 3DR Radio Telemetry are as follows:

- The 3DR Radio telemetry system serves as an open-source replacement for Xbee radios, offering a cost-effective alternative with enhanced range and performance.
- **Frequency:** Available in 433 MHz.

Configurations:

- Serial Board: For the air (mounted on the drone).
- USB Module: For the ground (connected to the ground control station).

Key Features:

Wireless Communication:

- Operates as a 2-way half-duplex system, allowing data transmission in both directions, but not simultaneously.
- Utilizes a standard TTL UART interface, enabling straightforward integration with various devices and systems.
- 433MHz Frequency Range
- Receiver sensitivity to -117dBm
- Transmit power up to 20dBm (100mW)

3.2.8 Power Module XT60 90A Specifications

The visual details of Power Module is given in Figure 3.7. The specifications of Power Module are as follows:

- Max Input Voltage: 28V
- Max Current : 90A
- Rapidly switching on and off, storing energy in inductors or capacitors.
- Connector Type: 6-position Molex cable directly plugs into the APM/Pixhawk 'PM' (Power Module) connector
- Configured for a 5V ADC

Key Functions:

- **Voltage Regulation:** The power module provides a stable 5.3V supply to power the flight controller and other onboard electronics.
- **Current Sensing:** Monitors the current up to 90A, allowing for real-time current monitoring during flight.
- **Voltage Sensing:** Measures input voltage up to 28V to monitor battery health and usage.

Applications:

- **Powering Flight Controllers:** Direct connection to APM or Pixhawk flight controllers to supply power and monitor voltage/current.
- **Telemetry and Safety:** Enables real-time monitoring of battery voltage and current, essential for telemetry data and triggering failsafe conditions.

3.3 Integration of Components

3.3.1 Power Supply And Connectivity

Power Distribution Board (PDB):

The visual details of PDB is given in Figure 3.8.

- **Function:** The PDB distributes power from the main battery to various components of the drone, such as the ESCs (Electronic Speed Controllers), motors, and other electronics.
- **Connector:** Uses XT60 connectors for reliable and high-current power transfer.

Power Supply to Pixhawk:

- **Connection:** The Pixhawk flight controller receives power through an XT60 connector Power Module rated for 90A. This ensures stable power delivery, essential for controlling the drone's navigation and flight systems.
- **Voltage Regulation:** Power is regulated and monitored through the Pixhawk's Power Module, which provides consistent 5.3V output and monitors voltage/current for safe operation.

TeraRanger Rangefinder Power Supply:

- **Power Source:** A buck converter is used to step down the voltage from the main battery to the required voltage level for the TeraRanger rangefinder.
- **Function:** This ensures the rangefinder receives a stable and appropriate power supply, enabling accurate distance measurement and obstacle detection.

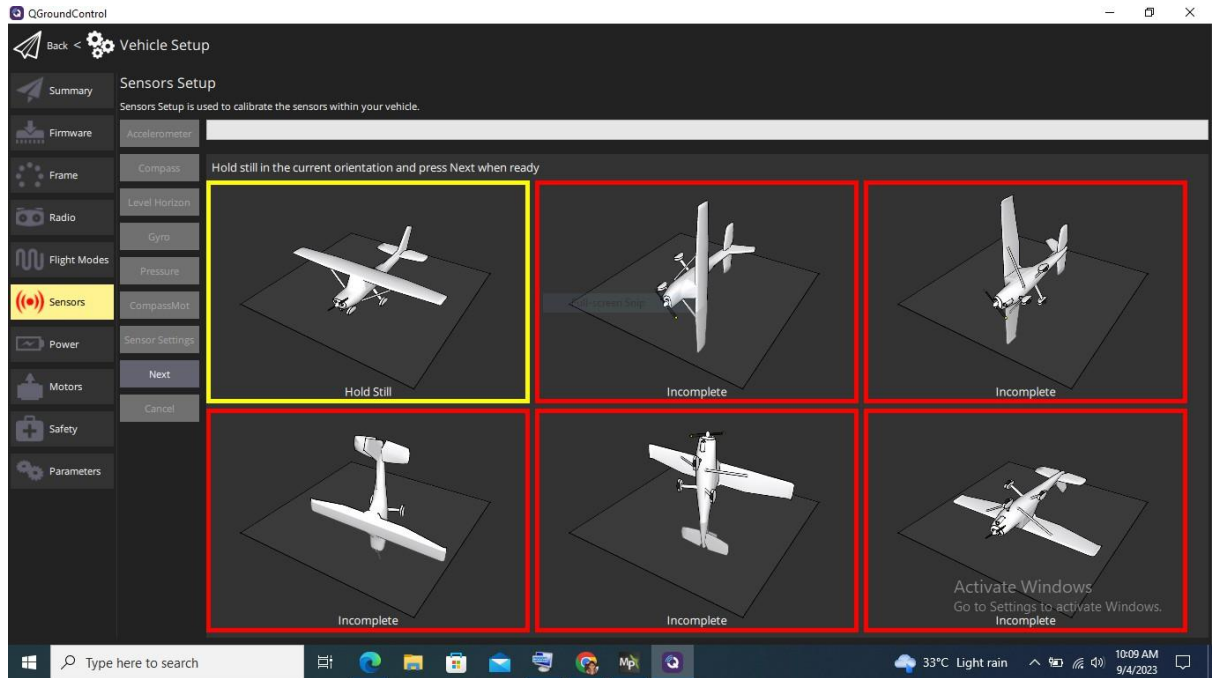


Figure 3.10: Firmware And Live Calibration.

3.3.2 Calibration, Firmware Script and Processing of Data

Flashing Firmware:

- The latest ArduPilot firmware or PX4 can be flashed onto the Pixhawk flight controller. This firmware is essential for enabling advanced autonomous features and ensuring compatibility with various sensors and peripherals.

Sensor Calibration:

- Calibration of key sensors like GPS, compass, accelerometer, gyro and Electronic Speed Controller (ESC) Calibration is performed to ensure accurate data readings. This process aligns the sensors with the drone's orientation and environment, enabling precise navigation and stability.

Parameter Configuration:

- Parameters can be provided into scripts or configured directly through the UI in QGroundControl or Mission Planner. These settings include essential flight characteristics like PID gains, flight modes, and fail-safes.

QGroundControl and Mission Planner Usage:

- QGroundControl and Mission Planner are comprehensive tools for managing drone operations:
- **QGroundControl:** Used for mission planning, sensor calibration, and parameter tuning. It provides a user-friendly interface for setting up waypoints, configuring flight modes, and real-time monitoring.
- **Mission Planner:** Another versatile tool similar to QGroundControl, offering additional scripting capabilities for advanced users.

3.3.3 Source Code Scripts:

The scripts of the source codes developed and deployed in the project are made publicly available on the following web links.

Source Code can Be find at: <https://github.com/isupercoder401/Agricultural-Drone-For-Crop-Monitoring-and-Early-Disease-Detection>

3.3.4 Circuit Diagram

Drone Circuitry:

METHODOLOGY

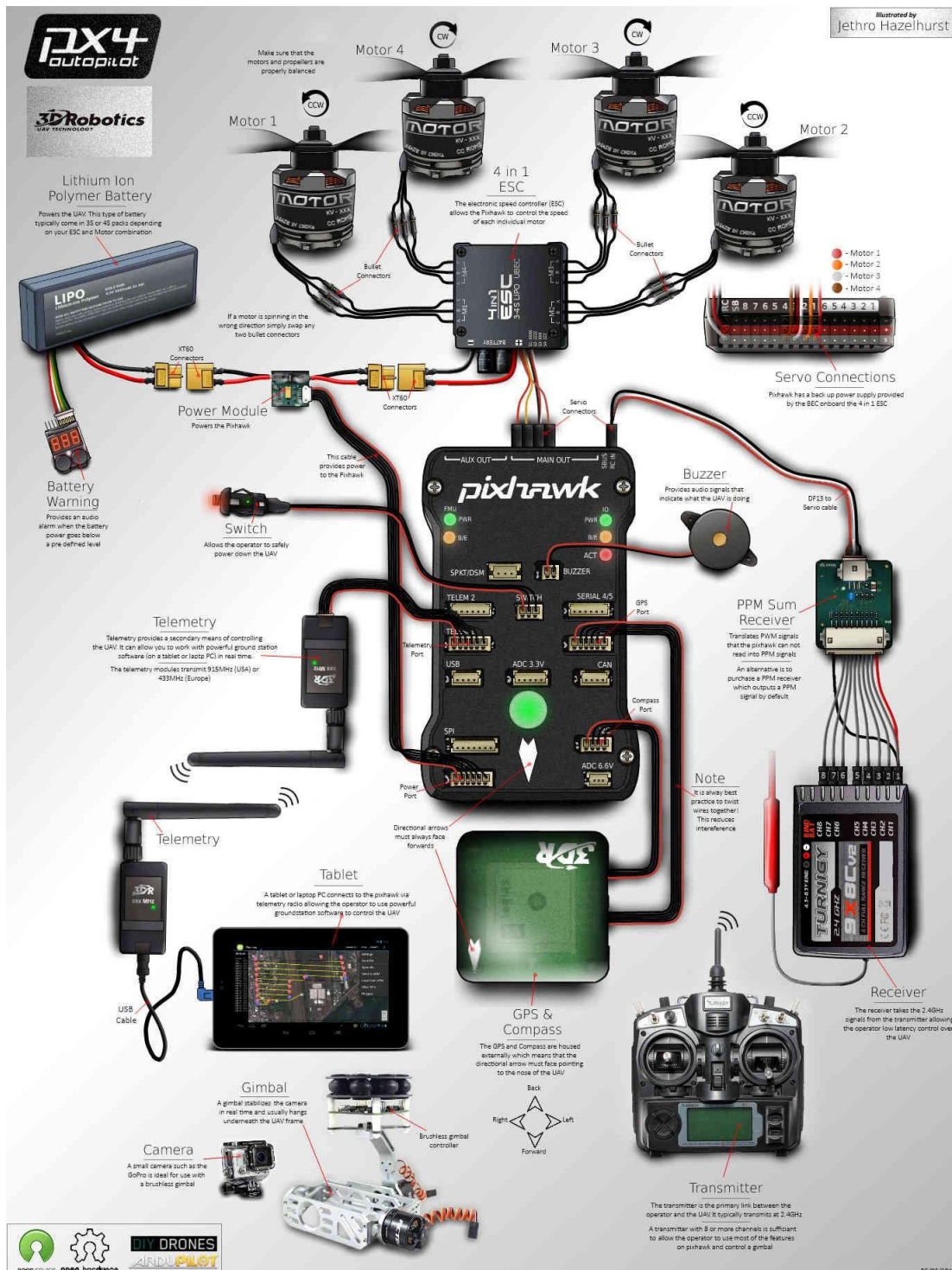


Figure 3.11: Drone Connection Diagram:

For more details, see link

https://docs.px4.io/main/assets/pixhawk_infographic2.B07zgBM6.jpg

Teraranger Connection:

Common GND pin 8 of teraranger with GND pin 4 of pixhawk and GND of Battery from below diagram.

EVO I2C backboard pins	Pixhawk I2C port pins
1 Tx	
2 Rx	
3 GND	
4 SDA	3 SDA
5 SCL	2 SCL
6	
7 VCC	1 VCC
8 GND	4 GND
9	

Figure 3.12: Teraranger connection with Pixhawk.

3.4 Software in the Loop

A simulation environment that allows you to test and debug your drone's software without the need for actual hardware. It is primarily used in conjunction with ArduPilot or PX4 autopilot software.

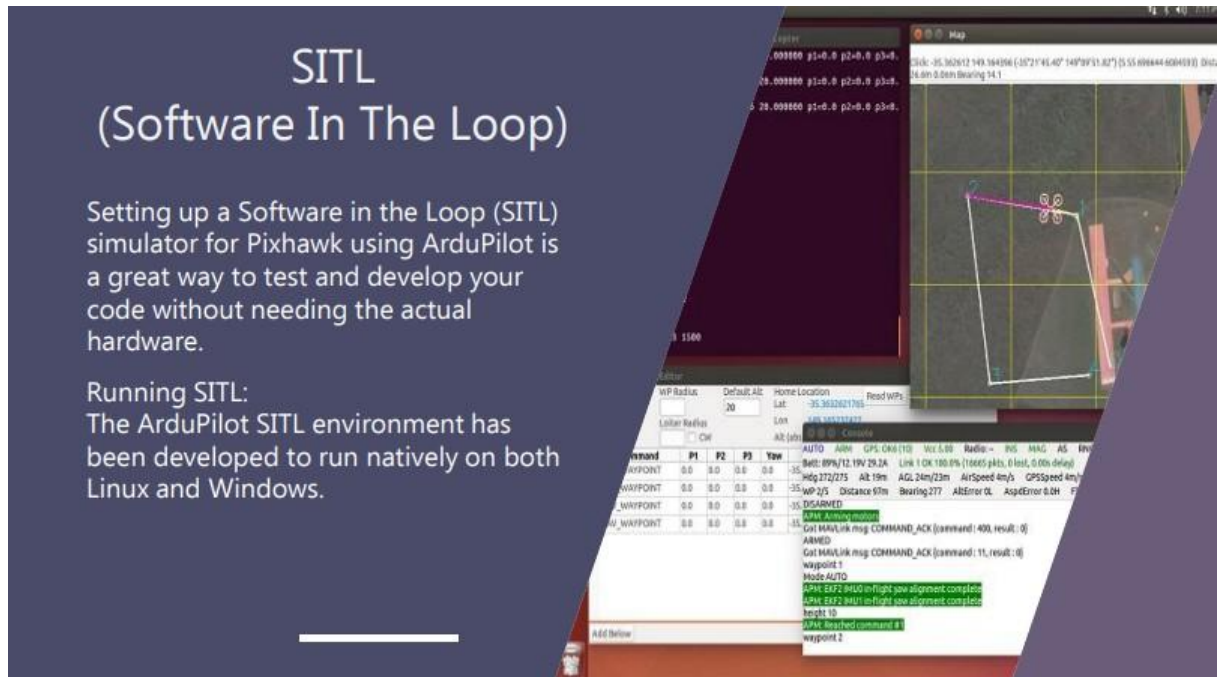


Figure 3.13: Software in the loop.

Setting up the Build Environment (Linux/Ubuntu)

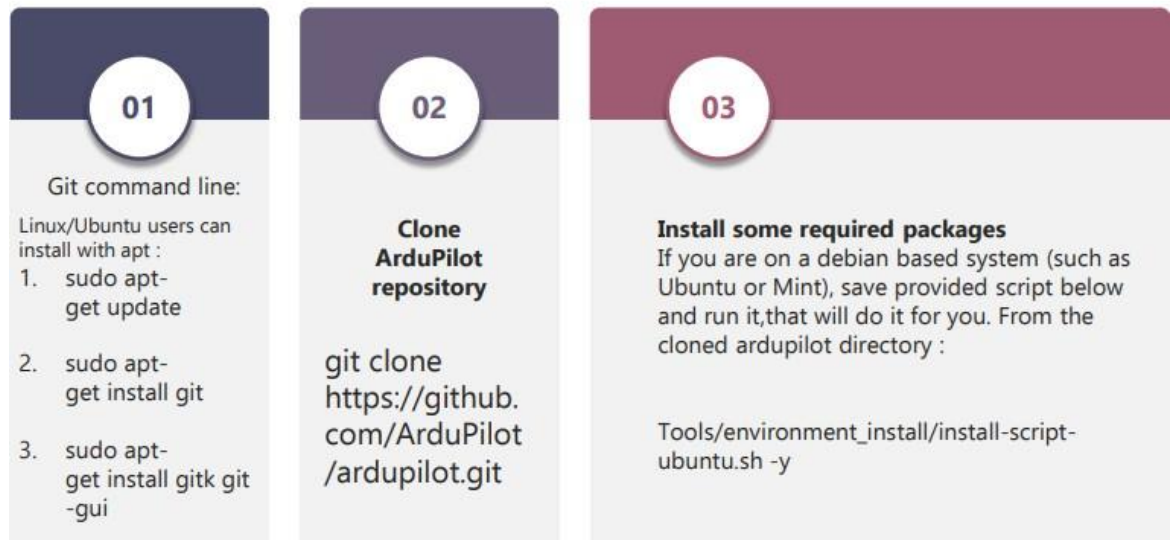


Figure 3.14: Build environment.

Source Script

<https://github.com/isupercoder401/Agricultural-Drone-For-Crop-Monitoring-and-Early-Disease-Detection>

Reload the path (log-out and log-in to make it permanent):
`. ~/.profile`

Start SITL simulator

1. `cd ardupilot/ArduCopter`
2. `sim_vehicle.py --console --map -w`

To get the most out of SITL you really need to use MAVProxy.....

1. `pip install --upgrade pymavlink MAVProxy --user`

FlightGear 3D View (Optional)

1. `sudo apt-get install flightgear`
2. Open a new command prompt and run the appropriate shell file for your vehicle in **/ardupilot/Tools/autotest/fg_quad_view.sh** (Copter)

Start SITL in the terminal in the normal way.....

1. `sim_vehicle.py -L KSFO`



Figure 3.15: FlightGear 3D.

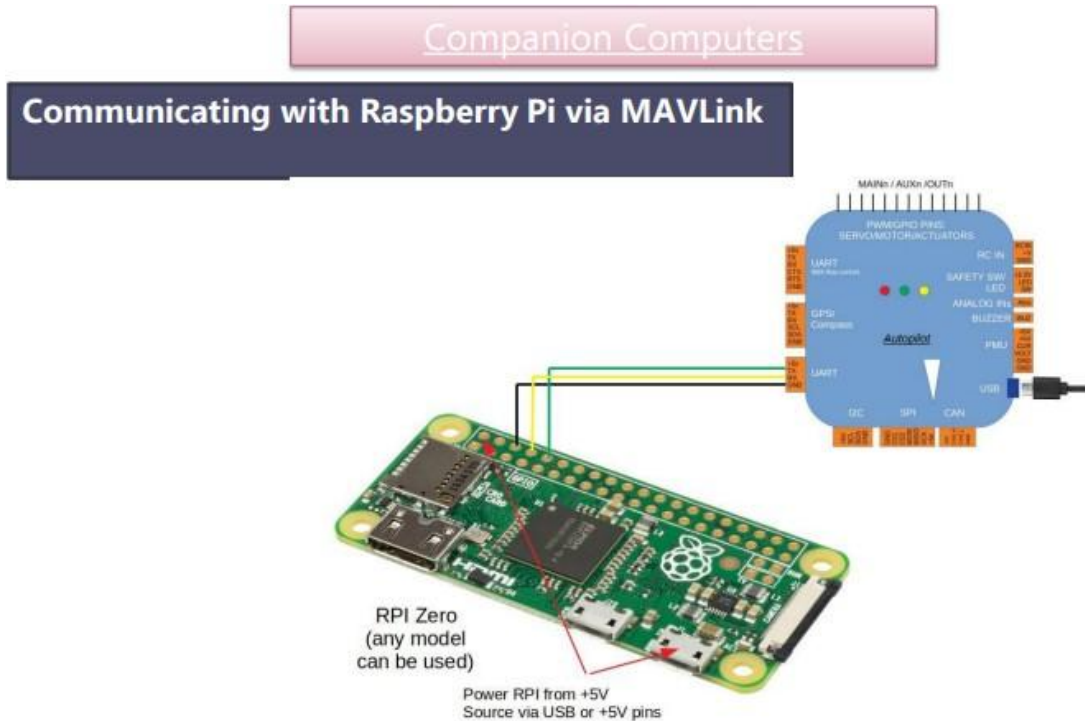


Figure 3.16: Rpi4 With Pixhawk.

Setting up the flight controller:

Connect to the flight controller and set the following parameters:
SERIAL2_PROTOCOL = 2 (the default) to enable MAVLink 2 on the serial port.
SERIAL2_BAUD = 921 so the flight controller can communicate with the RPi at 921600 baud.

LOG_BACKEND_TYPE = 3 if you are using APSync to stream the dataflash log files to the RPi

Configure the serial port (UART)¹

If not already configured, the Raspberry Pi's serial port (UART) will need to be enabled. Use the Raspberry Pi configuration utility for this.

Type:

```
sudo raspi-config
```

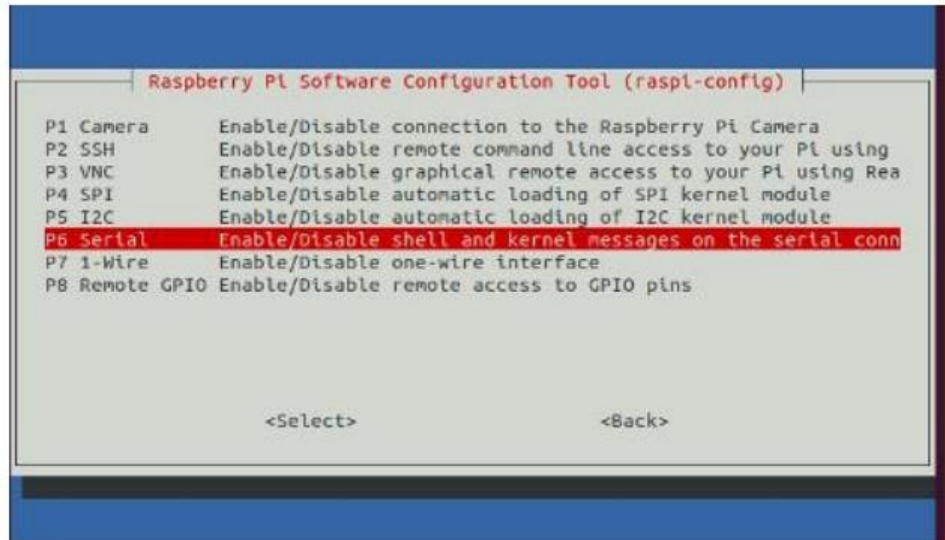


Figure 3.17: Enable Serial Port.

- `python3 mavproxy.py --master=/dev/serial0 --baudrate 921600 --aircraft MyCopter`
1. `param show ARMING_CHECK`
 2. `param set ARMING_CHECK 0`
 3. `arm throttle`

Figure 3.18: Mavproxy Commands.

Chapter 4

Implementation

In this chapter, we discuss the implementation details of autonomous agricultural drone project, focusing on the dataset collection process, the use of proposed TensorFlow model and TensorFlow Lite for edge devices, and the integration of YOLOv5 for real-time object detection.

4.1 Dataset Collection and Annotation

The project utilizes five specialized datasets, each tailored to a specific agricultural task [9]. The datasets description is given in Table 4.1.

Wheat Head Detection Dataset: used for identifying and counting wheat heads to estimate yield.

Crop vs Weed Detection Dataset: distinguishes between crops and weeds for precision weeding.

Disease Classification Dataset (38 Classes): identifies various plant diseases across different crops.

Wheat Disease Detection Dataset: focused on detecting diseases specific to wheat.

IMPLEMENTATION

Rice Disease Detection Dataset: identifies diseases in rice crops, expanding the system's applicability.

NAMES	FILES	CLASSES	NEED
Wheat Head Detection	• Training Set: 4855 images (77%) • Validation Set: 971 images (15%) • Test Set: 486 images (8%)	1. Another 2. Head 3. Null	The wheat head detection dataset is essential for estimating crop yield. By accurately counting wheat heads in the field, farmers can predict harvest quantities and make better decisions regarding harvest timing and resource allocation.
Weed vs. Crop Detection	• Training Set: 3432 images • Validation Set: 2221 images	1. Weed 2. Crop	The crop vs. weed detection dataset is necessary for developing models that can distinguish between beneficial crops and harmful weeds. This capability is vital for automated weeding systems that can reduce herbicide usage and protect crops.

IMPLEMENTATION

Plant Disease Classification	• Training Set: 70,295 images across 38 classes • Validation Set: 33 images across 38 classes • Test Set: 17,572 images	38 classes 1. Apple__Apple_scab 2. Apple__Black_rot 3. Apple__Cedar_apple_rust 4. Apple__healthy 5. Blueberry__healthy 6. Cherry_(including_sour)__Powdery_mildew 7. Cherry_(including_sour)__healthy 8. Corn_(maize)__Cercospora_leaf_spot 9. Corn_(maize)__Common_rust 10. Corn_(maize)__Northern_Leaf_Blight 11. Corn_(maize)__healthy 12. Grape__Black_rot 13. Grape__Esca_(Black_Measles) 14. Grape__Leaf_blight_(Isariopsis_Leaf_Spot) 15. Grape__healthy 16. Orange__Haunglongbing_(Citrus_greening) 17. Peach__Bacterial_spot 18. Peach__healthy 19. Pepper,_bell__Bacterial_spot 20. Pepper,_bell__healthy 21. Potato__Early_blight 22. Potato__Late_blight 23. Potato__health 24. Raspberry__healthy	The plant disease and wheat disease datasets provide the foundation for real-time monitoring of crop health. Early detection of diseases can prevent their spread, protect yields, and ensure food security.
-------------------------------------	--	--	---

IMPLEMENTATION

		25. Soybean___healthy 26. Squash___Powdery_mildew 27. Strawberry___Leaf_scorch 28. Strawberry___healthy	
--	--	--	--

		29. Tomato___Bacterial_spot 30. Tomato___Early_blight 31. Tomato___Late_blight 32. Tomato___Leaf_Mold 33. Tomato___Septoria_leaf_spot 34. Tomato___Spider_mites Two-spotted_spider_mite 35. Tomato___Target_Spot 36. Tomato___Tomato_Yellow_Leaf_Curl_Virus 37. Tomato___Tomato_mosaic_virus 38. Tomato___healthy	
--	--	--	--

IMPLEMENTATION

Wheat Disease Detection	• Training Set: 6107 images (82%) • Validation Set: 802 images (11%) • Test Set: 501 images (7%)	1. Brown Rust 2. Healthy 3. Powdery Mildew 4. Septoria 5. Stem Rust	ANNOTATIONS This dataset is mini version of Plant Disease Classification Dataset and is self annotated using Labelbox, only a few disease parts in every wheat plant.
Rice Disease Detection	• Training Set: 34,445 images (80%) • Validation Set: 4284 images (10%) • Test Set: 4305 images (10%)	1. tungro 2. bacterial-blight 3. bacterial-leaf-streak 4. blast 5. brown-spot	The rice disease detection dataset adds diversity to the system, ensuring that the drone can be deployed in various crop types beyond wheat. This adaptability makes the drone a versatile tool in different agricultural settings. ANNOTATIONS This dataset comes with pre-trained annotated files.

Figure 4.1: Datasets description is given in this Table.

4.1.1 Training Process and Parameters

Our tensor flow model was trained using disease classification dataset (38 Classes) using the below parameters and steps:

Link for Complete Code is given as: <https://github.com/isupercoder401/Agricultural-Drone-For-Crop-Monitoring-and-Early-Disease-Detection>

Google Drive

- The first step is to mount/connect Google Drive to access the *checkpoint dir* where the trained model checkpoints will be stored. This is done using the `drive.mount` command.

Setting Up Checkpoints

- The script creates a directory for storing checkpoints if it doesn't already exist.
- The ModelCheckpoint callback is defined to save model weights at different epochs during training. The weights are saved as .h5 files in the checkpoint directory.

Data Preparation

- The dataset is loaded using

```
tf.keras.preprocessing.image_dataset_from_directory
```

for training, validation, and testing. The images are resized to 224x224 pixels, and the labels are set to categorical mode.

- The datasets are then prefetched using

```
tf.data.AUTOTUNE
```

to improve performance.

Building the Model

- The model is based on EfficientNetB7, a pre-trained model from TensorFlow's applications library. The top layers are excluded

```
(include_top = False)
```

```
(trainable = False)
```

, typically, fine-tuning involves unfreezing and retraining some layers of the model while keeping others frozen. However, with trainable=False, no layers are updated.

IMPLEMENTATION

- A GlobalAveragePooling2D layer is added to reduce the dimensions of the features extracted by EfficientNetB7.
- A final dense layer with 38 output neurons is used to classify the images into one of the 38 categories using softmax activation.

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
efficientnetb7 (Functional)	(None, 7, 7, 2560)	64,097,687
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2560)	0
dense_1 (Dense)	(None, 38)	97,318

Total params: 64,195,005 (244.88 MB)

Trainable params: 97,318 (380.15 KB)

Non-trainable params: 64,097,687 (244.51 MB)

Model Compilation

- Model is compiled using Categorical Cross Entropy as the loss function, Adam as optimizer, and accuracy as the metric.

Training the Model

- Two callbacks, **EarlyStopping** and **ReduceLROnPlateau**, are implemented to monitor and improve the training process [6].
- EarlyStopping stops training if the validation loss doesn't improve for three consecutive epochs.
- ReduceLROnPlateau reduces the learning rate if the validation loss stops improving.

The model is trained for up to 50 epochs with the defined callbacks and validation data.

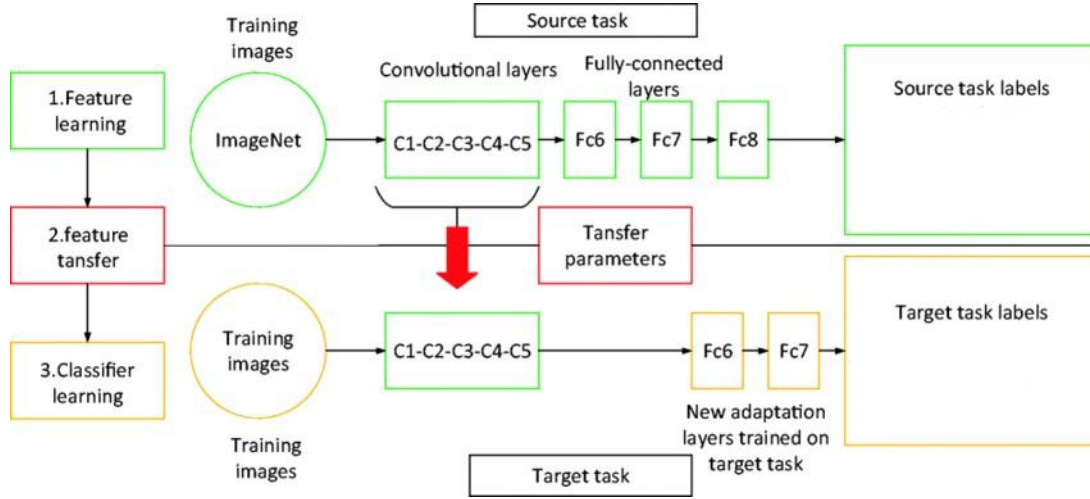


Figure 4.2: Flow-Chart

4.1.2 TensorFlow Lite Conversion

After training, the model is converted to TensorFlow Lite (TFLite) format, making it lightweight and suitable for edge devices. The TFLite model is saved to a specified path.

4.1.3 Prediction and Evaluation

- Predictions are made using the trained model. The **argmax** function is used to determine the predicted class.
- The classification report is generated using

sklearn.metrics.classification_report

and an F1 score is calculated for each class.

4.1.4 Visualizing the F1 Scores

- A bar chart is plotted to visualize the F1 scores for each class. This helps in understanding which classes are performing well and which are not.

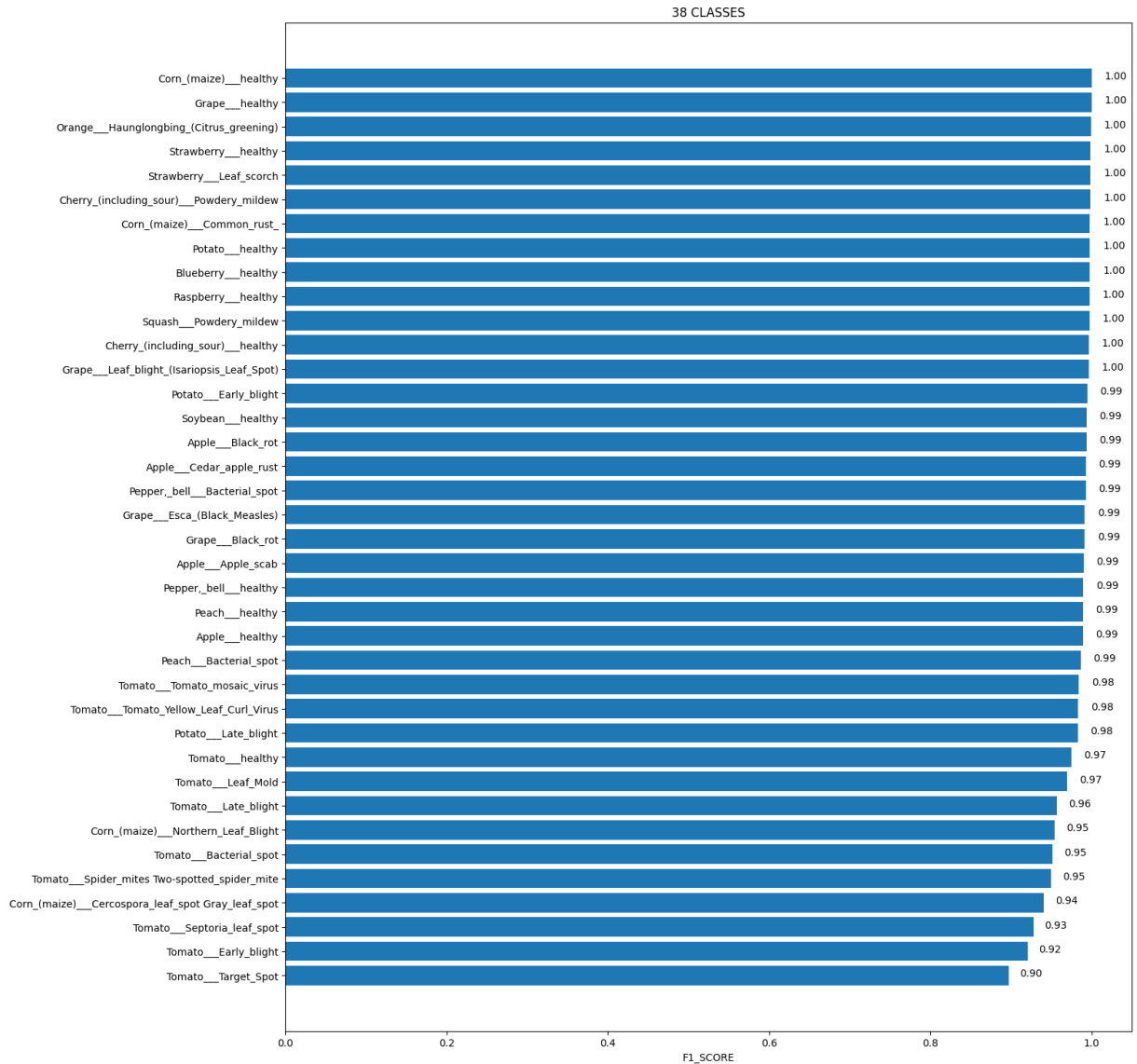


Figure 4.3: Bar Chart

4.1.5 Custom Image Loading and Prediction

- A custom function is created to randomly load an image from a directory, preprocess it, and make a prediction using the trained model.
- The actual and predicted classes, along with the prediction probability, are displayed on the image.

4.1.6 Visualizing Wrong Predictions

- The script visualizes the images that were misclassified by the model. For each of these images, it shows the actual class, predicted class, and the prediction probability.

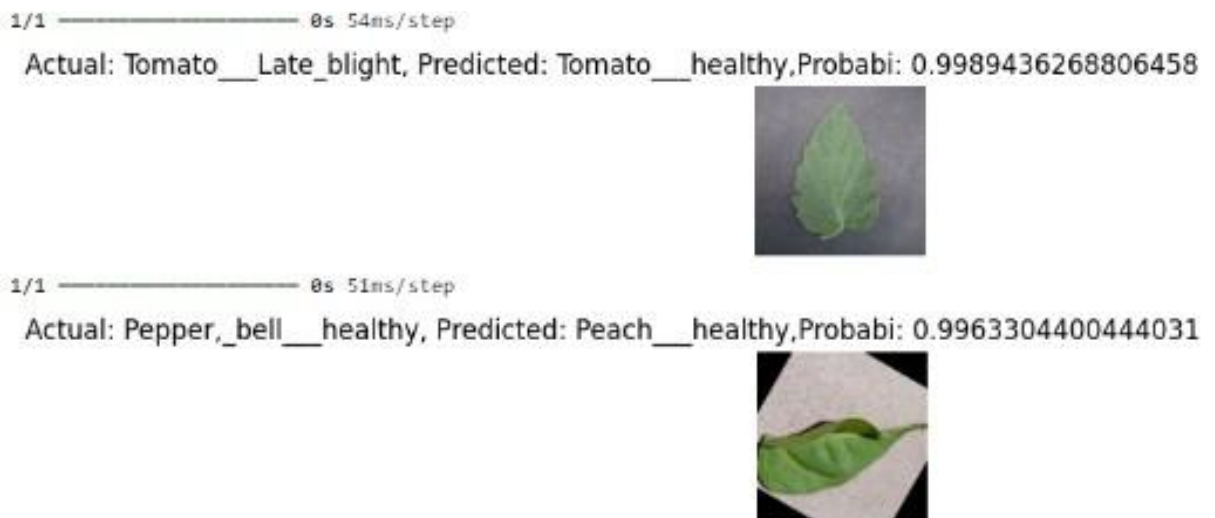
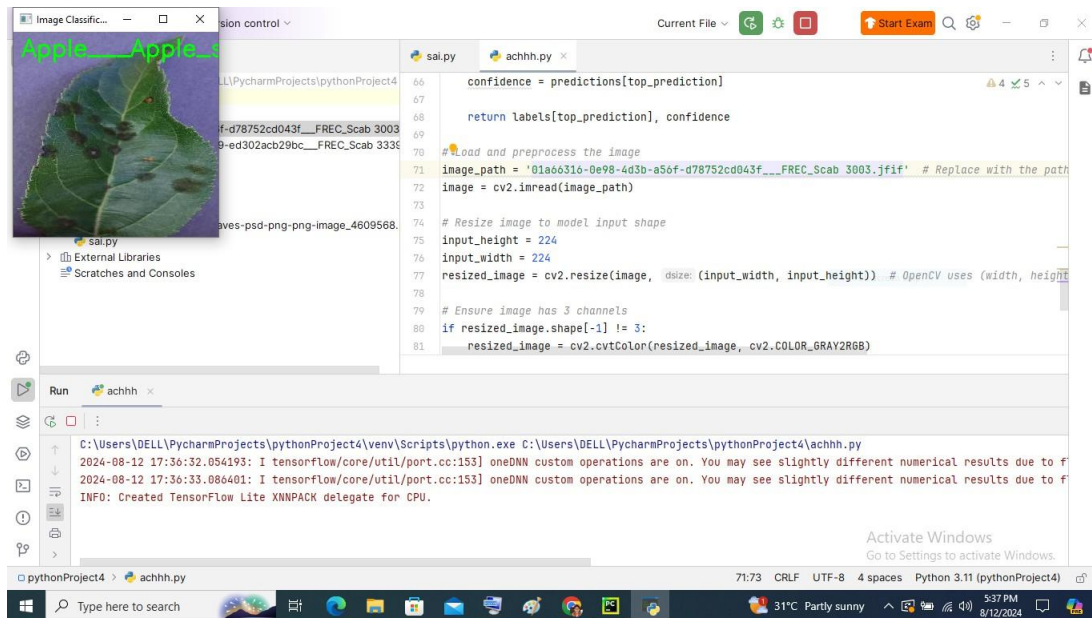


Figure 4.4: Visualizing Wrong Predictions With Highest Probability



Figure 4.5: Visualizing Wrong Predictions With Highest Probability

4.1.7 Predictions Using Live Camera



Code link for real-time inference using drone camera is given as:

<https://github.com/isupercoder401/Agricultural-Drone-For-Crop-Monitoring-and-Early-Disease-Detection>

Figure 4.6: Real-time Inference

4.2 Training Using Yolov5

4.2.1 Flow For A YOLOv5 Object Detection Model

Dataset Preparation

- **Annotations and Images:** The dataset consists of images and corresponding annotation files (usually in .txt format) that contain the bounding box labels for objects within the images.
- **Data Organization:** The images and labels are organized into directories, typically separated into train, val, and test folders. The labels use a format where each line represents one object, specifying the class ID and the bounding box coordinates ($center_x$, $center_y$, $width$, $height$) normalized be-

tween 0 and 1. The data.yaml file is a configuration file used in YOLOv5 to define the paths to your dataset, no. classes, and their names. This file is crucial for training, validating, and testing your YOLOv5 model as it tells the model where to find the data and what classes to expect.

Model Configuration

- **YOLOv5 Model Selection:** The specific YOLOv5 model variant (e.g. **yolo v5n**, **yolov5s**, **yolov5m**, **yolov5l**, or **yolov5x**) is chosen based on the desired balance between speed and accuracy [7]. I chose yolov5n (nano) for Wheat Head Detection Dataset and yolov5m for Rice, Wheat Disease Detection Datasets.
- **Configuration File:** A **YAML configuration file** is used to define parameters such as the number of classes, dataset paths, and model architecture.

Training the Model

- **Hyperparameters:** Training hyperparameters are set, including lr, batch size, epochs, image size, and augmentation techniques (like flipping, scaling, and color adjustments). I chose 640x640x3 and 224x224x3 two different image sizes and number of epochs to be 99 and 29 for different datasets depending upon my need.

Training Process: The model iterates over the training dataset, updating its weights to minimize the difference between bounding boxes and the ground truth boxes using loss functions for objectness, bounding box coordinates, and class confidence. **Validation:** During training, the model's performance is evaluated on the validation data for progress and avoid overfitting. Results are continuously saved to **runs** folder and weights in **weights** folder after every epoch.

Model Evaluation

- **Metrics Calculation:** After training, the model is evaluated on the test set using metrics such as Precision, Recall, mAP (mean Average Precision),

and F1 score.

- The model predicts bounding boxes and class labels for objects in the test images. Non-Maximum Suppression (NMS) is applied to remove overlapping boxes.

4.3 Deploying a YOLOv5Model on a Raspberry Pi4

Download Best.pt weights file after training completed and save it on server or github or export in usb [10].

In Rpi4:

1. **Install Dependencies**
2. **Install Python Libraries**
3. **Clone YOLOv5 Repository**
4. **Clone or copy best.pt to Raspberry Pi**
5. **Move best.pt to the YOLOv5 directory**
6. **Connect the Camera**
7. **Run the YOLOv5 Model inference on the Camera Feed**

Link for Complete code for above steps is given as :

<https://github.com/isupercoder401/Agricultural-Drone-For-Crop-Monitoring-and-Early-Disease-Detection>

Chapter 5

Experimental Results

5.1 Experimental Setup And Performance Metrics

The dataset is divided into training and validation sets. Performance metrics and losses are calculated at each epoch. Box loss, objectness loss, classification loss for both training and validation sets, precision, recall, and mAP values.

- **Precision:** This metric indicates the percentage of correctly identified positive predictions among all the instances the model labeled as positive. A high precision value means the model makes few incorrect positive predictions.
- **Recall:** This metric reflects the proportion of actual positive instances that the model successfully recognized. A high recall value signifies that the model misses few actual positive cases.
- **Training Box Loss:** Training Box Loss represents the discrepancy between the predicted bounding boxes and the actual ground-truth bounding boxes during model training.
- **Training objectness Loss:** Refers to the error or discrepancy between the predicted objectness scores (probabilities that a specific region contains an object) and the actual ground-truth labels (indicating whether the region contains an object or is just background) during the training process.

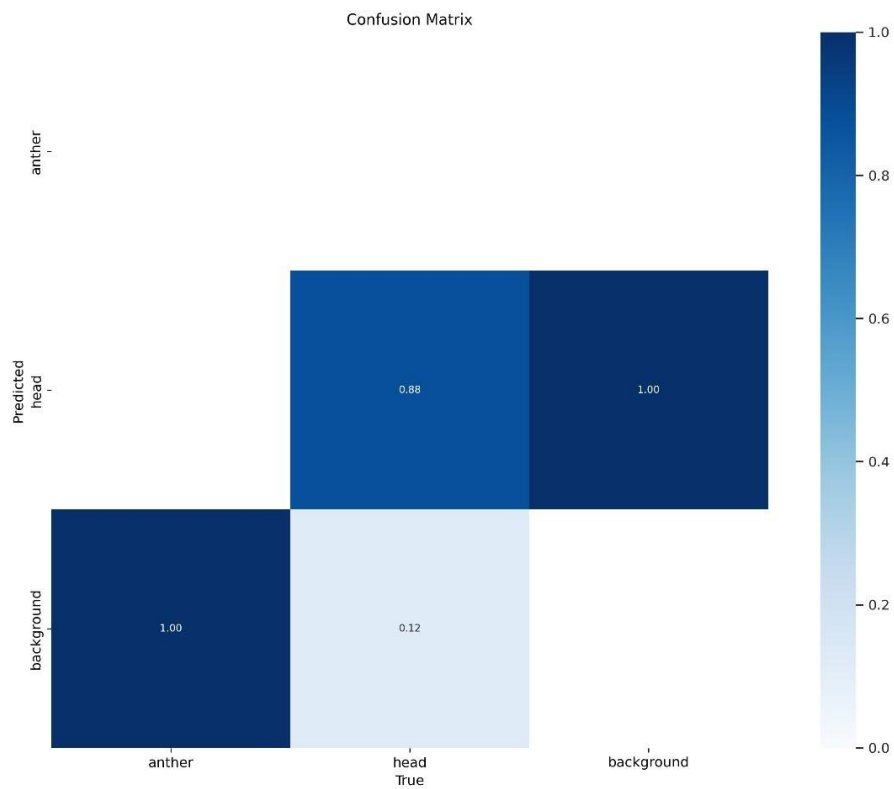
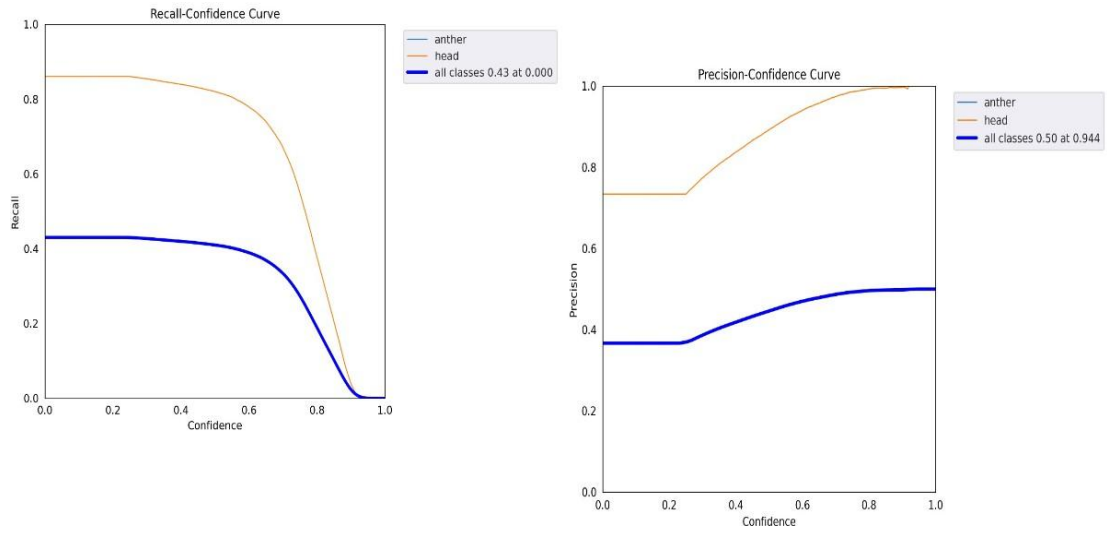
- **Training Classification Loss:** Refers to the error or discrepancy between the predicted class labels for objects within the bounding boxes and the actual ground-truth class labels during the training process.
- **Validation Box Loss:** Validation Box Loss refers to the error between the predicted bounding boxes and the actual ground-truth bounding boxes during the model testing on validation data.
- **Validation objectness Loss:** Refers to the error or discrepancy between the predicted objectness scores (probabilities that a specific region contains an object) and the actual ground-truth labels (indicating whether the region contains an object or is just background) during the model testing on validation data.
- **Validation Classification Loss:** Refers to the error or discrepancy between the predicted class labels for objects within the bounding boxes and the actual ground-truth class labels during the model testing on validation data

5.2 Results Analysis

Significant improvements were observed in the precision and recall over epochs. The mAP metrics showed a steady increase indicating better detection capabilities. A decreasing trend was observed in both training and validation losses showing model improvements and generalization. For Diseases Classification Tf Model code proposed in this project provides: accuracy is 97.07 and val accuracy is 98.02 while val loss is 0.05 at learning rate: 5.0000e-04 till 50 epochs [11].

EXPERIMENTAL RESULTS

WHEAT HEAD DETECTION MODEL



EXPERIMENTAL RESULTS

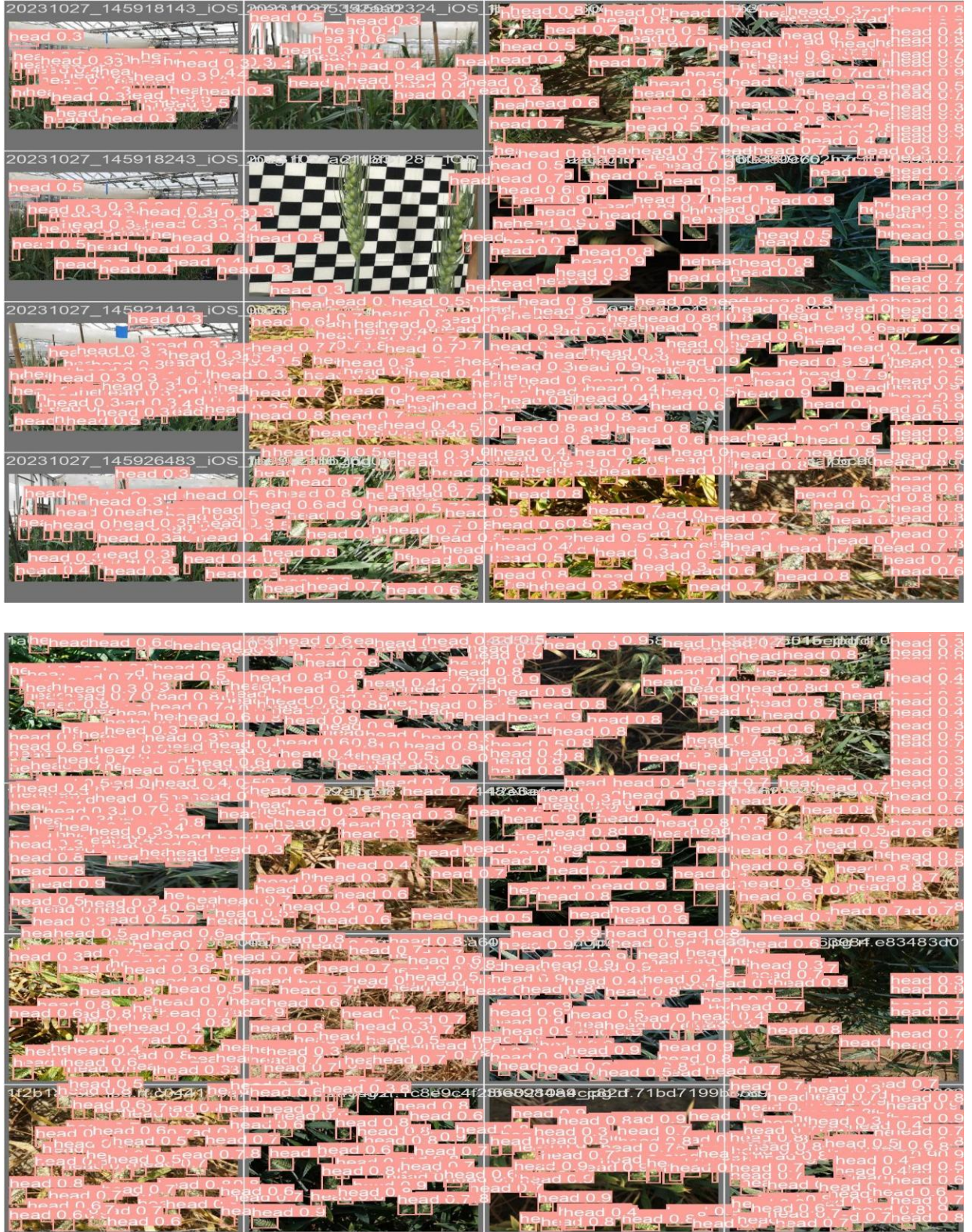


Figure 5.1: Real-Time Predictions Using YOLOv5 Model

EXPERIMENTAL RESULTS

RICE DISEASE DETECTION MODEL

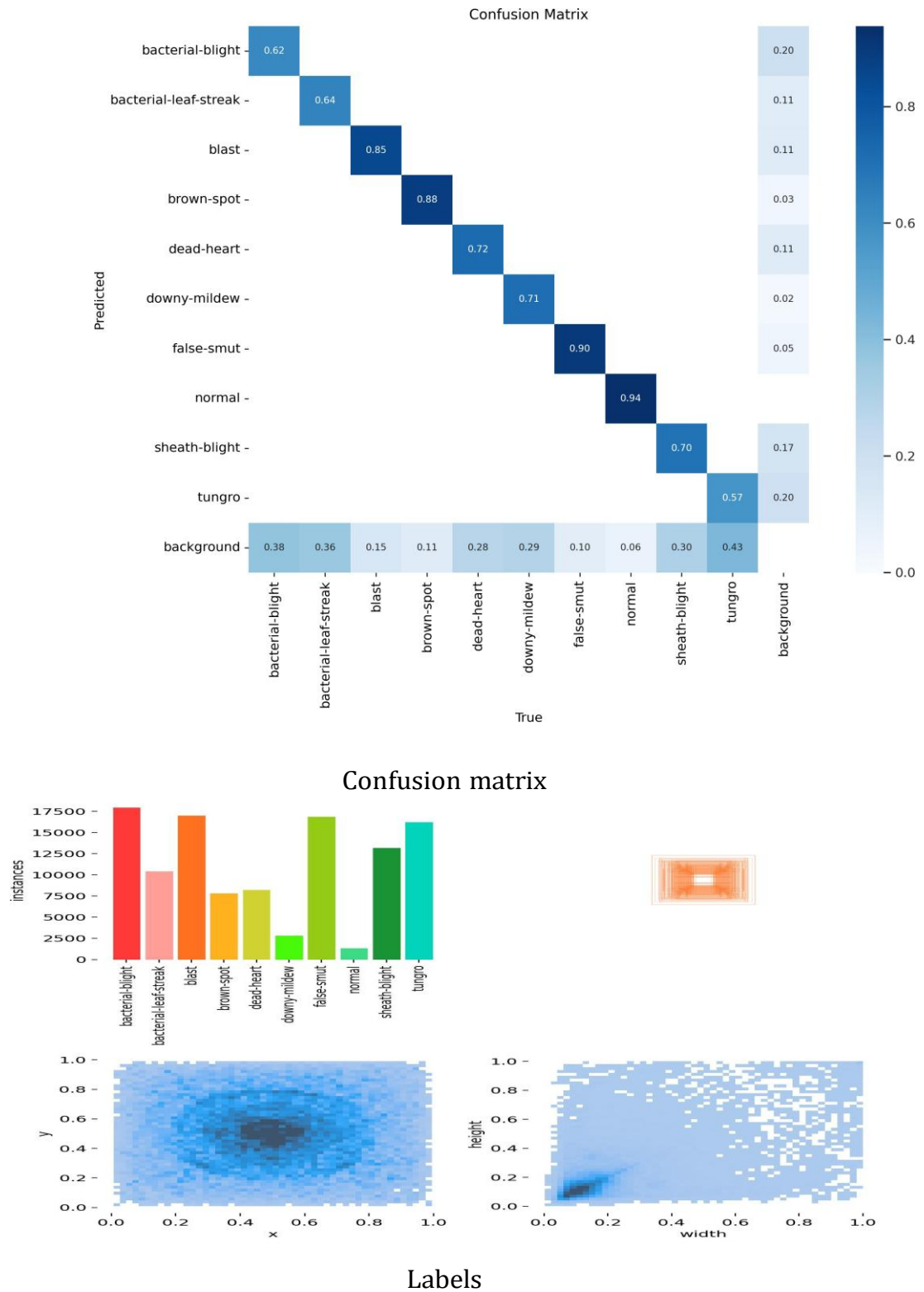


Figure 5.2: Real-Time RICE Disease Detection

EXPERIMENTAL RESULTS

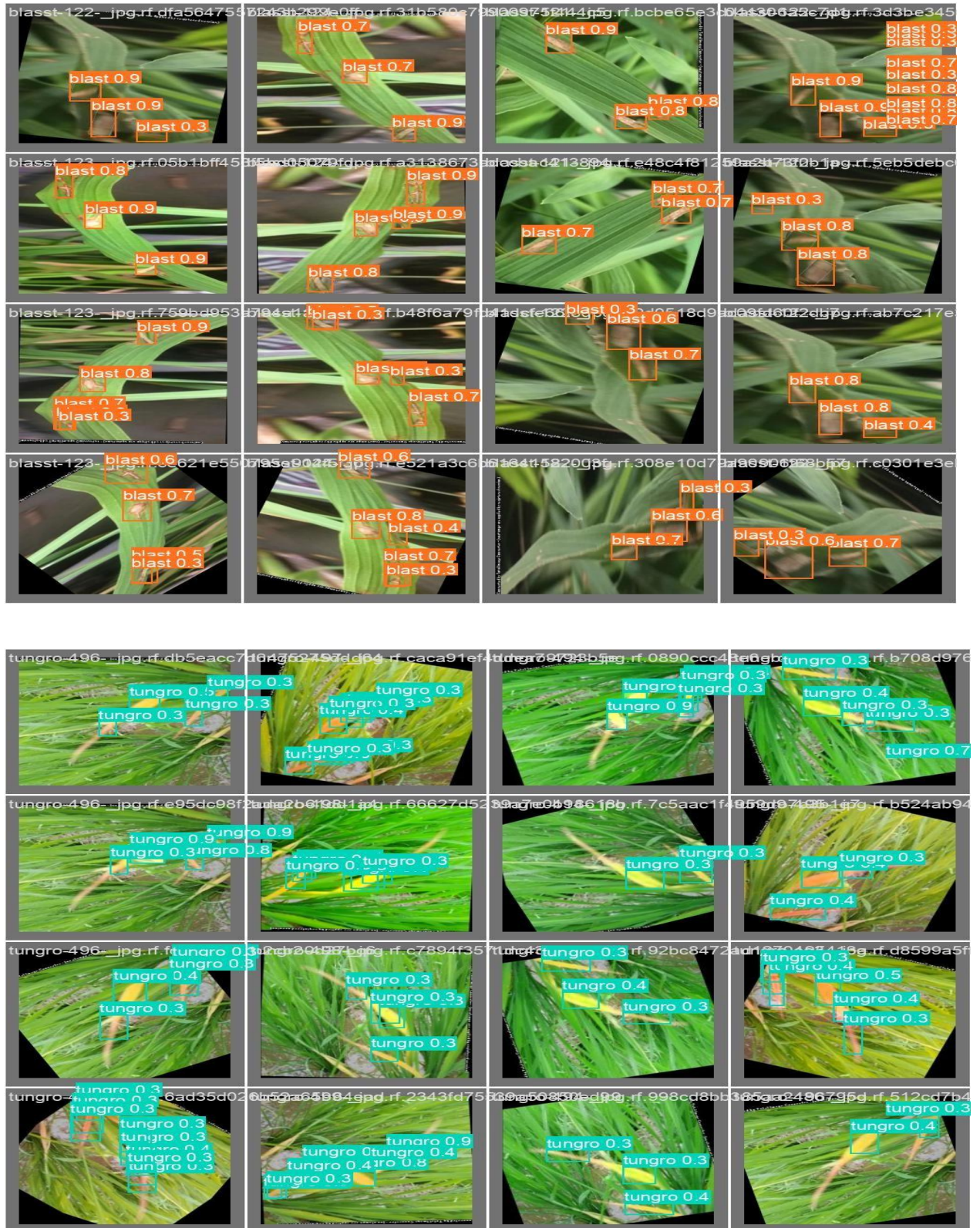


Figure 5.4: Real-Time RICE DISEASE DETECTION

Weed Vs Crop Model

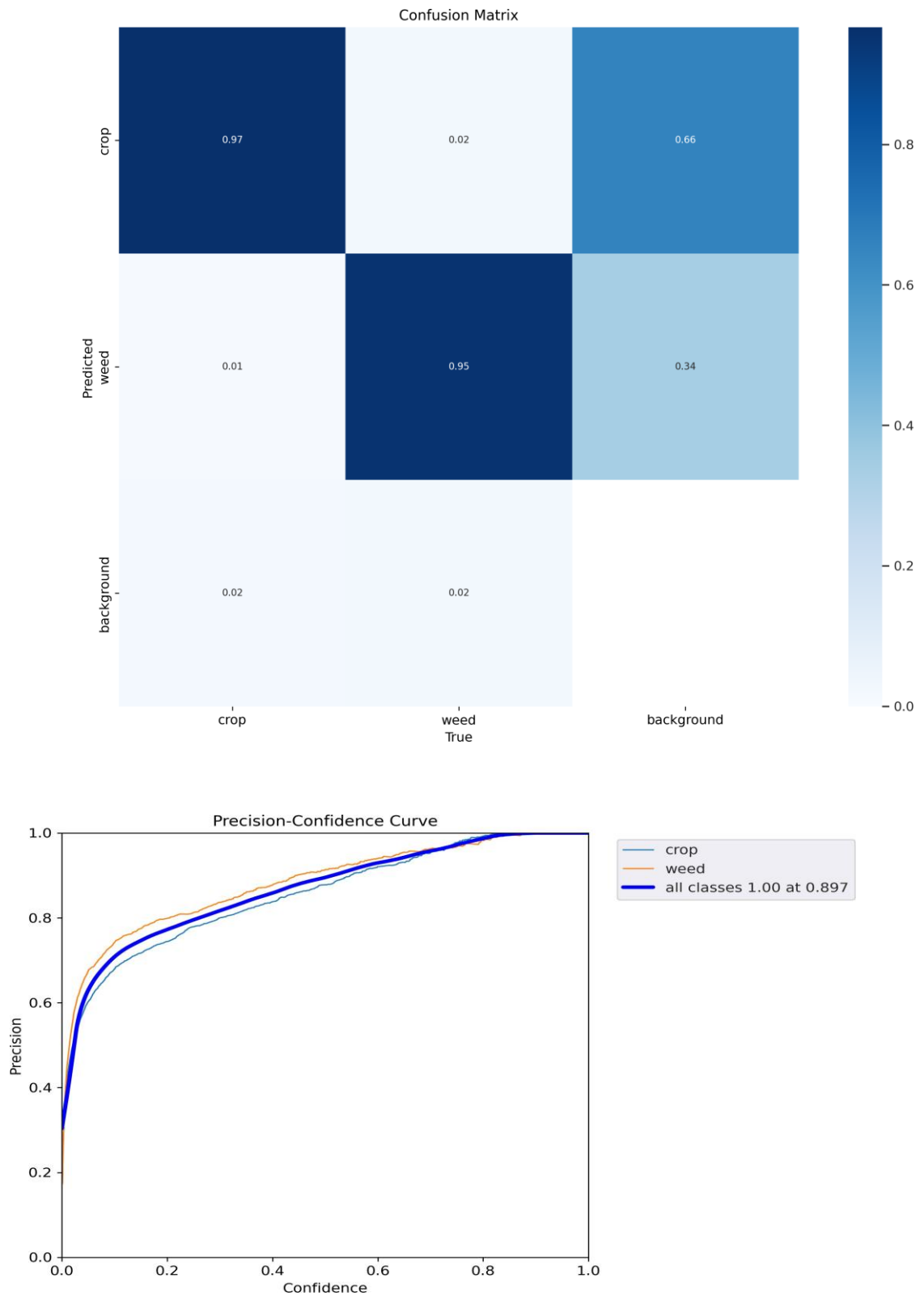


Figure 5.5: Real-Time Weed Vs Crop Detection

EXPERIMENTAL RESULTS

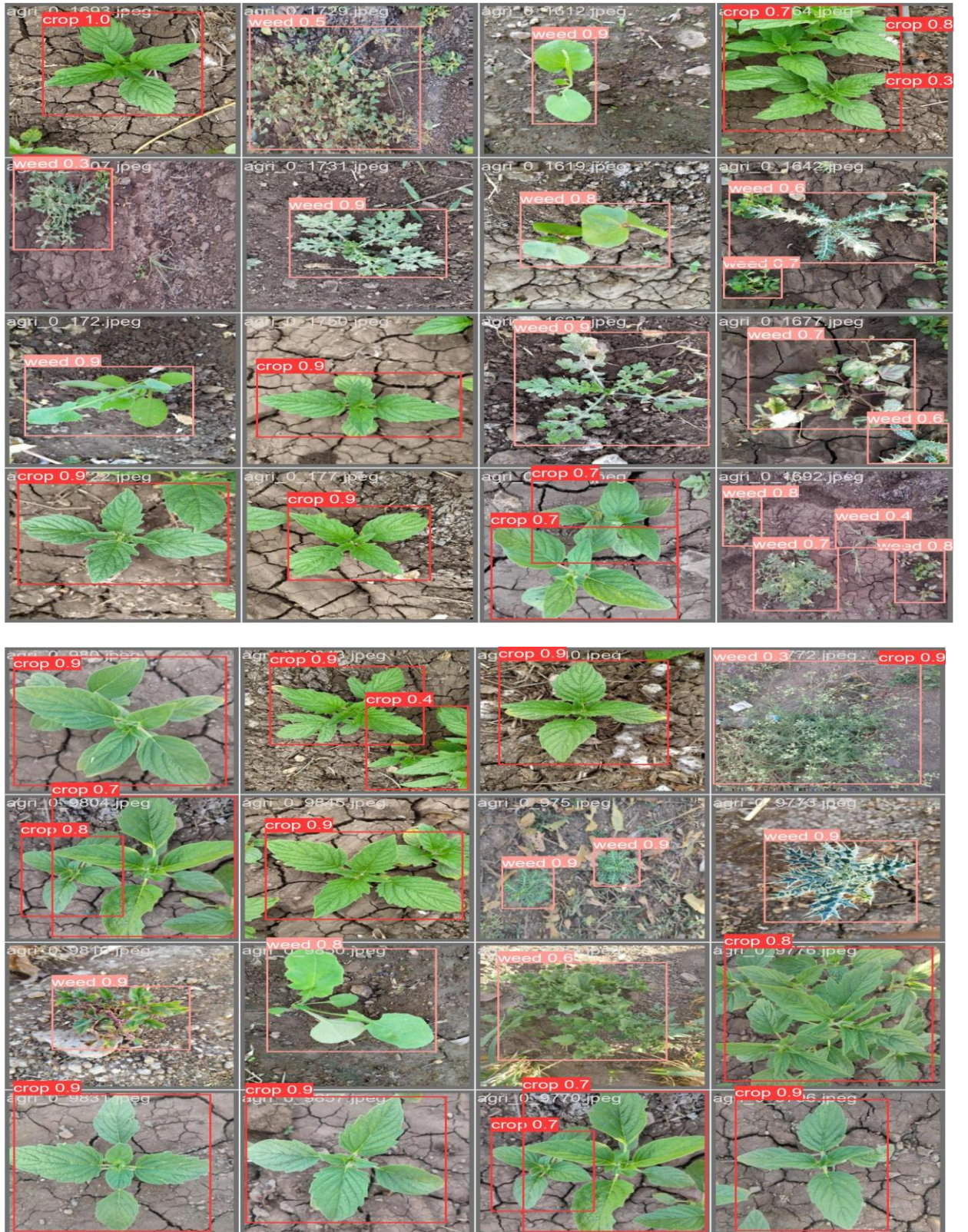


Figure 5.6: Real-Time Weed Vs Crop Detection

Wheat Disease Detection

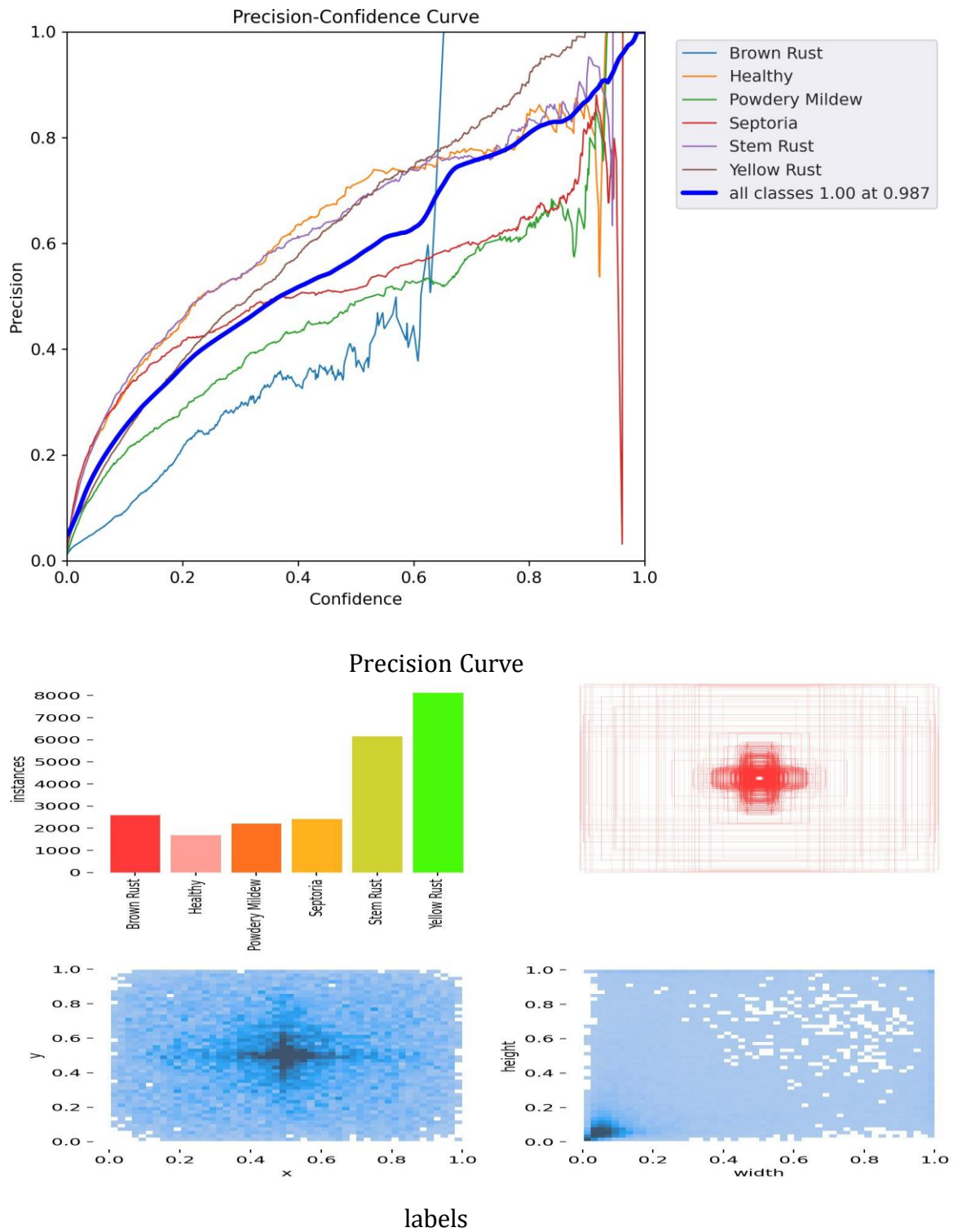
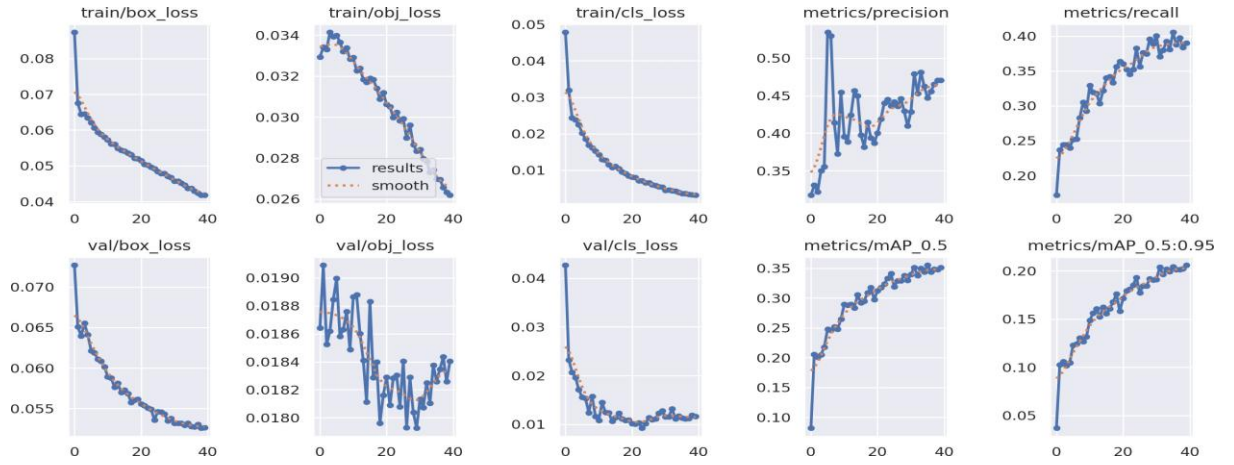


Figure 5.7: Wheat Disease Detection

EXPERIMENTAL RESULTS



results



Real-Time Prediction

Figure 5.8: Wheat Disease Detection

5.3 Discussion

The development and implementation of the autonomous agricultural drone for crop and weed detection, disease identification, and wheat head detection yielded several key **insights and interpretations**, which are discussed below.

5.3.1 Effectiveness of Datasets

The datasets, tailored for specific agricultural tasks such as weed vs. crop detection, disease classification, and wheat head identification, were effective in training the TF and YOLOv5 models. These datasets contained images representing various crops, weeds, and diseases, helping the model generalize well.

5.3.2 Model Performance

The model demonstrated a remarkable improvement in performance across the training epochs, as indicated by the recorded metrics. By the conclusion of the training process, the model attained high precision and recall values, showcasing its capability to detect and classify crops, weeds, and diseases within the agricultural environment.

5.3.3 Hardware-Software Integration

The success of the project was heavily dependent on the seamless integration of the drone's hardware components (Pixhawk flight controller, Raspberry Pi 4, BLDC motors, GPS, compass, and TeraRanger range finder, camera and many more) with the software stack (ROS, MAVLink, OpenCV, Sift, Ardupilot, Px4, TF and YOLOv5). This integration enabled the drone to perform real-time object detection, navigation, and precision control. The effective communication between these components allowed for autonomous flight operations and accurate detection and classification of agricultural features [12].

5.4 Challenges and Limitations

5.4.1 Dataset Limitations

While the datasets were effective for training, their limited size and low quality and few annotated data, the controlled nature of the images may not fully capture the variability of real-world agricultural environments. The diversity of crop fields and diseases and the wide range of conditions in which the drone may operate could require additional training on more extensive and varied datasets to enhance the model's robustness and generalization.

5.4.2 Hardware Constraints

Although the computational power of the Raspberry Pi 4 was sufficient for this project, it may be inadequate for more complex or large-scale agricultural deployments. Processing higher-resolution images or handling more intricate detection tasks might necessitate more powerful hardware. Furthermore, the physical capabilities of the drone, such as the precision of the BLDC motors, less powerful battery and the range of the sensors, could limit its ability to perform tasks that require higher accuracy or strength, such as detailed crop surveillance or large-scale field coverage. Also need of multispectral camera for disease detection and proper compact frame is required to make it suitable even in rainy weather.

5.4.3 Real-Time Processing

Achieving real-time processing with high accuracy remains a challenge, particularly in the dynamic and unpredictable conditions of agricultural fields. Balancing processing speed with model accuracy is crucial, especially when managing large data streams from the camera and sensors while navigating complex environments. Latency in communication between components, particularly in transmitting large amounts of data, can affect the drone's responsiveness and overall performance.

5.4.4 Safety and Reliability

Ensuring the safety and reliability of the drone in real agricultural operations is critical. Any malfunction or incorrect detection could result in damage to crops or the drone itself. Comprehensive testing and validation are required to ensure that the drone performs reliably in a variety of conditions before it is deployed in real-world farming scenarios.

5.5 Summary

In conclusion, the development of the autonomous agricultural drone demonstrated the practicality of using TF, YOLOv5, Sift, Mavlink, dronekit, mavproxy and Ardupilot for real-time object detection and navigation in agricultural scenarios. The insights gained from this project provide a strong foundation for further research and development in the field of precision agriculture. Addressing the identified challenges and limitations will be vital for advancing this technology and ensuring its effectiveness and reliability in real-world applications.

Chapter 6

Conclusion

6.1 Summary of Findings

The development and deployment of the Autonomous Agricultural Drone for smart crop monitoring and early disease detection have provided significant insights into the potential of integrating advanced object detection algorithms with agricultural technologies. The key findings from the project are summarized below:

6.1.1 Effective Object Detection

The TF and YOLOv5 models successfully detected various agricultural features, including crops, weeds, and diseases, achieving high precision and recall. The use of datasets tailored for specific agricultural tasks enabled the model to effectively distinguish between different classes, such as healthy crops, weeds, and diseased plants. This capability is crucial for targeted interventions in precision agriculture.

6.1.2 Real-Time Performance

The autonomous agricultural drone demonstrated exceptional real-time performance across various operational tasks. The drone equipped with failsafe features, auto-landing and RTL capabilities, ensuring safe and reliable opera-

tion. These features allowed the drone to autonomously return to a designated home location and land safely in the event of low battery, signal loss, or completion of a mission. Auto PID (Proportional-Integral-Derivative) tuning was implemented to optimize the drone's flight stability and responsiveness. This automated process adjusted the drone's control parameters in real-time, ensuring smooth and stable flight under varying conditions. The project utilized QGroundControl and Mission Planner for mission planning, sensor calibration, and parameter configuration. These ground control stations provided a user-friendly interface for designing complex flight paths, monitoring real-time telemetry, and executing autonomous missions.

6.1.3 Challenges and Limitations

The project encountered several challenges, including dataset limitations, hardware constraints, real-time processing demands, and varying environmental conditions. Addressing these challenges is essential for enhancing the robustness and reliability of the drone in real-world agricultural applications.

6.2 Future Work

While the current project has laid a strong foundation for smart agricultural drones, several areas for future work and improvements have been identified [13]:

6.2.1 Dataset Expansion

Expanding the dataset to include a wider variety of real-world agricultural scenarios, such as different crop types, growth stages, and environmental conditions, would improve the model's robustness and generalization. Collaborating with agricultural research institutions and farmers to obtain real-world data and augmenting the dataset with synthetic data from various simulations could be beneficial.

6.2.2 Enhanced Hardware

Upgrading the computational hardware to more powerful platforms, such as the latest NVIDIA GPUs or TPUs, would allow for processing higher-resolution images and more complex scenes. Additionally, enhancing the drone's physical components, such as using more precise navigation sensors, battery and stronger motors, would improve its flight stability and operational capabilities.

6.2.3 Advanced Algorithm

Implementing more advanced algorithms for real-time processing, such as optimizing the YOLOv5 model and YOLOv8 for faster inference, would further enhance the system's performance. Exploring other object detection models like EfficientDet or integrating complementary models with YOLOv5 could also provide additional strengths in accuracy and speed.

6.2.4 Autonomous Decision-Making

Advancing the autonomous decision-making capabilities of the drone by incorporating machine learning algorithms for predictive analytics and situational awareness would improve its effectiveness in agricultural management. This could involve integrating reinforcement learning techniques to optimize the drone's actions based on real-time data, leading to more efficient and intelligent crop monitoring and management.

Bibliography

- [1] Sindhuja Sankaran, Lav R Khot, Cesar Z Espinoza, and Shayan Jarolmasjed. Low-altitude, high-resolution aerial imaging systems for row and field crop phenotyping: A review. *European Journal of Agronomy*, 70:112–123, 2015.
- [2] Anne-Katrin Mahlein. Plant disease detection by imaging sensors—parallels and specific demands for precision agriculture and plant phenotyping. *Plant Disease*, 100(2):241–251, 2016.
- [3] Ning Zhang, Michael Wang, and Ning Wang. Precision agriculture—a worldwide overview. *Computers and Electronics in Agriculture*, 36(2-3):113–132, 2002.
- [4] Andreas Kamilaris and Francesc X Prenafeta-Boldú. Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147:70–90, 2018.
- [5] Eyal David and Marcel Serouart. A review of uav monitoring in precision agriculture. *Science of The Total Environment*, 739:139092, 2020.
- [6] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke,

- Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: Large-scale machine learning on heterogeneous systems. <https://www.tensorflow.org/>, 2015.
- [7] Glenn Jocher, Ayush Chaurasia, Jin Qiu, and Alexy Stoken. Yolov5. <https://github.com/ultralytics/yolov5>, 2020. Accessed: 2024-07-20.
- [8] PX4 Development Team. Pixhawk flight controller documentation, 2021.
- [9] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.
- [10] Morgan Quigley, Brian Gerkey, Ken Conley, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y Ng. Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3, 2009.
- [11] Mark Everingham, S M Ali Eslami, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes challenge: A retrospective. *International journal of computer vision*, 111(1):98–136, 2015.
- [12] Dimosthenis C Tsouros, Saliha Bibi, and Panagiotis G Sarigiannidis. A review on uav-based applications for precision agriculture. *Information*, 10(11):349, 2019.
- [13] Liang Zhou, Wei Zhang, Zexuan Yang, Dengpan Zhang, and Yutian Wei. Recent advances in uav-based remote sensing for precision agriculture. *Computers and Electronics in Agriculture*, 191:106567, 2022.